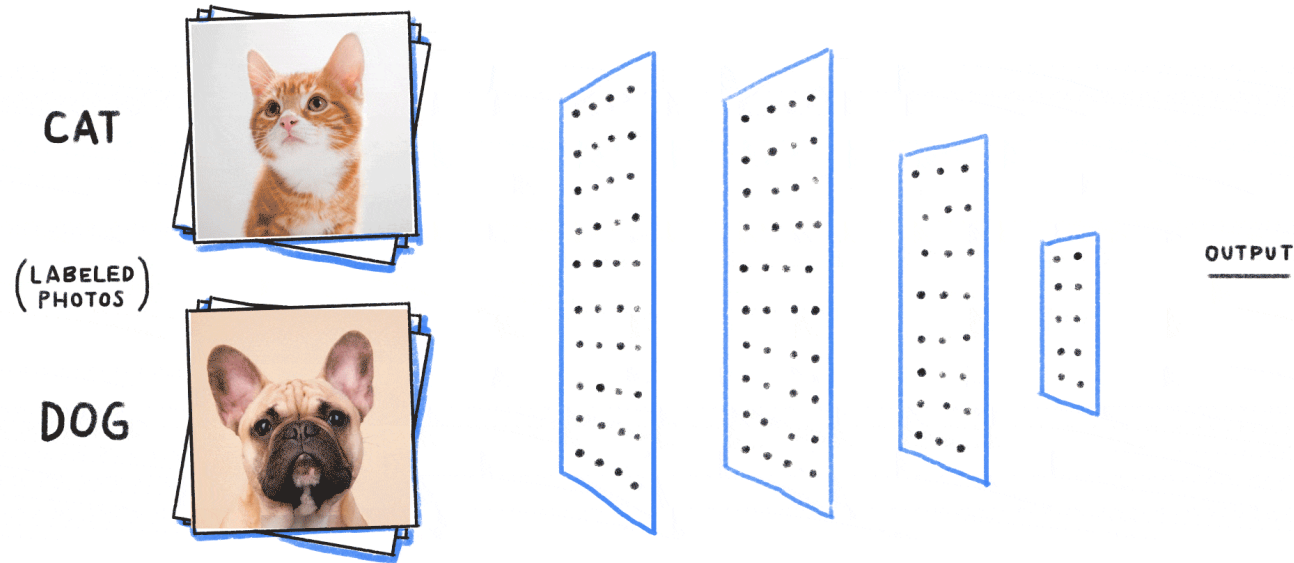


REDES CONVOLUCIONALES

Ignacio Gómez
2025

Redes Neuronales Convolucionales



From [becomingahuman](#)

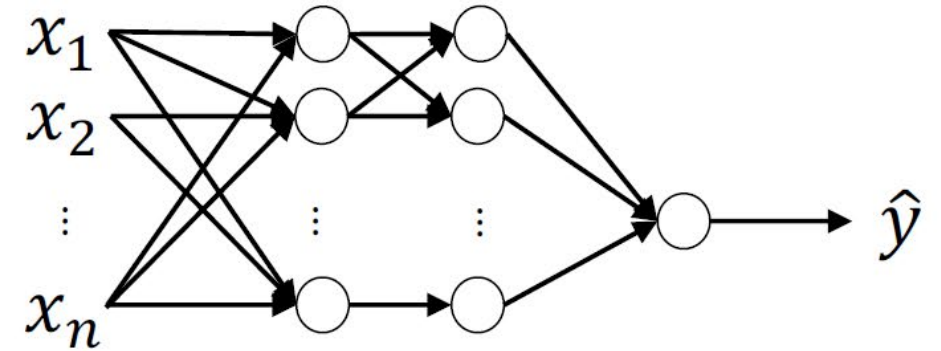
Una red convolucional (o CNN, del inglés Convolutional Neural Network) es un tipo de red neuronal diseñada específicamente para procesar datos estructurados en forma de rejilla, como imágenes, donde las relaciones espaciales entre los datos son importantes. Las CNN son muy populares en tareas de procesamiento de imágenes y visión por computadora, como clasificación, detección de objetos y segmentación.

Redes Neuronales con imágenes grandes

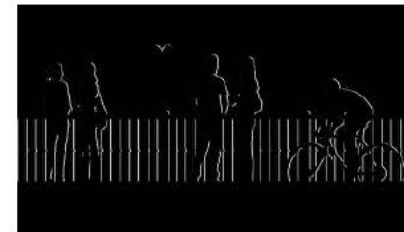
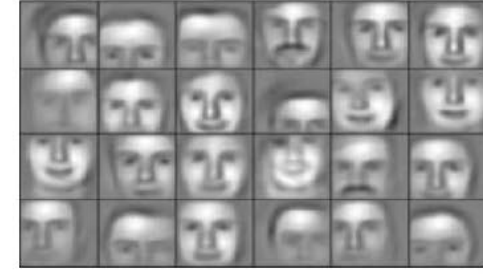
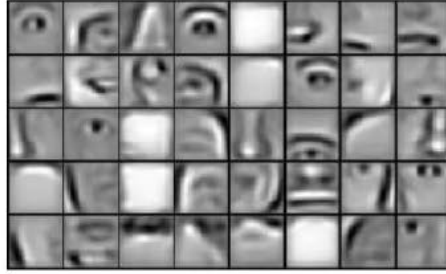
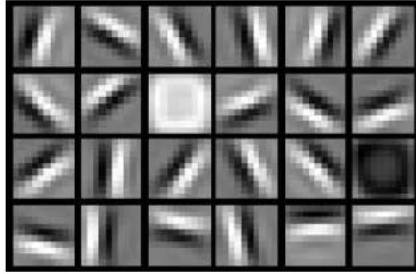


64x64

→ Cat? (0/1)



Filtros en tratamiento de imágenes



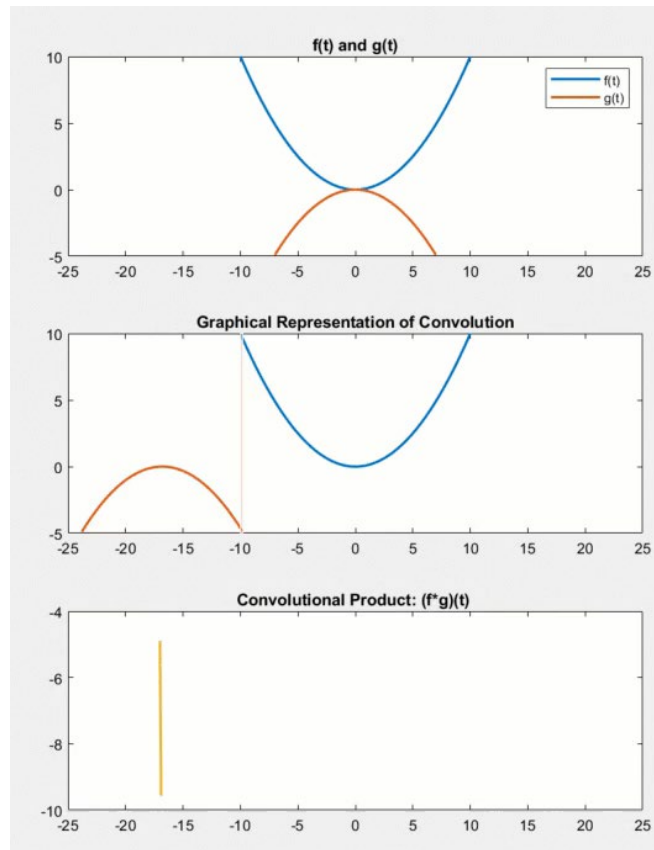
vertical edges



horizontal edges

Capas en una Red Neuronal Convolutiva.

- **Capas Convolucionales:** Son el núcleo de las CNN. Aplican filtros (o kernels) mediante convoluciones a la entrada para extraer características locales, como bordes, texturas y patrones.



From [mathworks](#)

La convolución es una operación matemática que combina dos funciones para describir la superposición entre ambas. La convolución toma dos funciones, “desliza” una sobre la otra, multiplica los valores de las funciones en todos los puntos de superposición, y suma los productos para crear una nueva función.

Matemáticamente se describe como:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

Filtro o Kernel Convolutacional

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Filtro o Kernel Convolutacional

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Filtro o Kernel Convolutacional

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Filtro o Kernel Convolutacional

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Filtro o Kernel Convolutacional

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Filtro o Kernel Convolutacional

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Filtro o Kernel Convolutacional

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

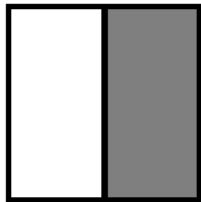
*

=

0	-2	-4	-7
-3	-2	-3	-16

Filtro de detección de aristas verticales

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	<u>10</u>	<u>10</u>	<u>0</u>	0	0



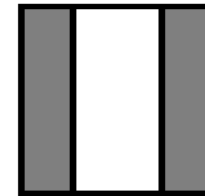
*

1	0	-1
1	0	-1
1	0	-1

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

*



Aristas verticales claro-oscuro y viceversa

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

1	0	-1
1	0	-1
1	0	-1



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10



*

1	0	-1
1	0	-1
1	0	-1



=

0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0



Filtro convolucional con parámetros entrenables

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

=

0	-2	-4	-7
-3	-2	-3	-16

Convolución en el contexto de las CNN

Matriz con los
datos de entrada

4	2	6	4	0	8	0	3
1	1	0	7	5	0	4	2
0	3	1	2	5	5	1	0
3	5	0	2	1	1	7	1
0	5	0	0	1	5	6	4
7	0	8	0	3	1	3	9
3	1	3	0	6	4	2	2
0	5	1	0	1	0	0	3

Una imagen representada como una matriz 2D (en escala de grises) o 3D (con canales de color, como RGB).

Filtro/kernel de
la convolución

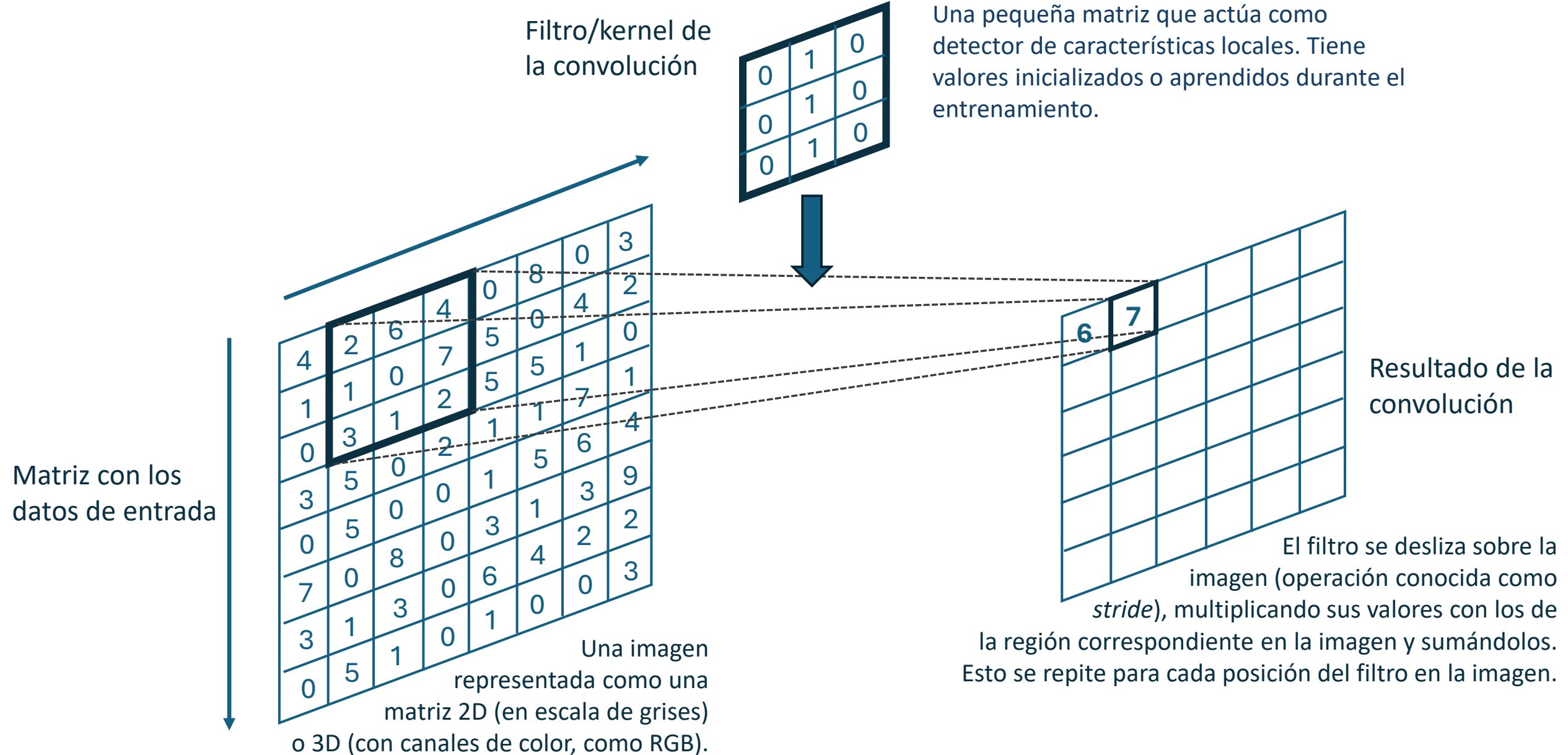
0	1	0
0	1	0
0	1	0

Una pequeña matriz que actúa como detector de características locales. Tiene valores inicializados o aprendidos durante el entrenamiento.

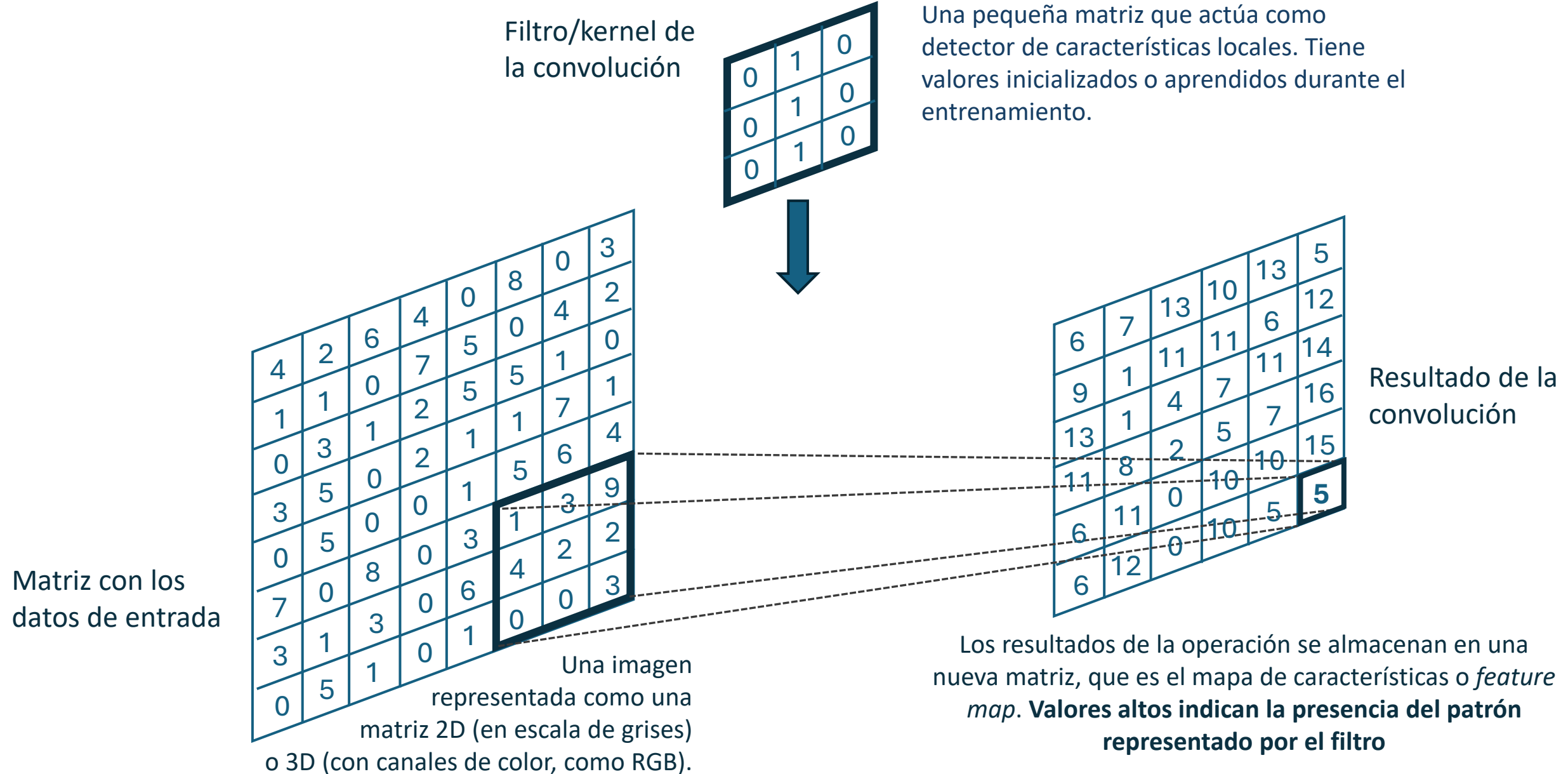
Resultado de la
convolución

6

Convolución en el contexto de las CNN

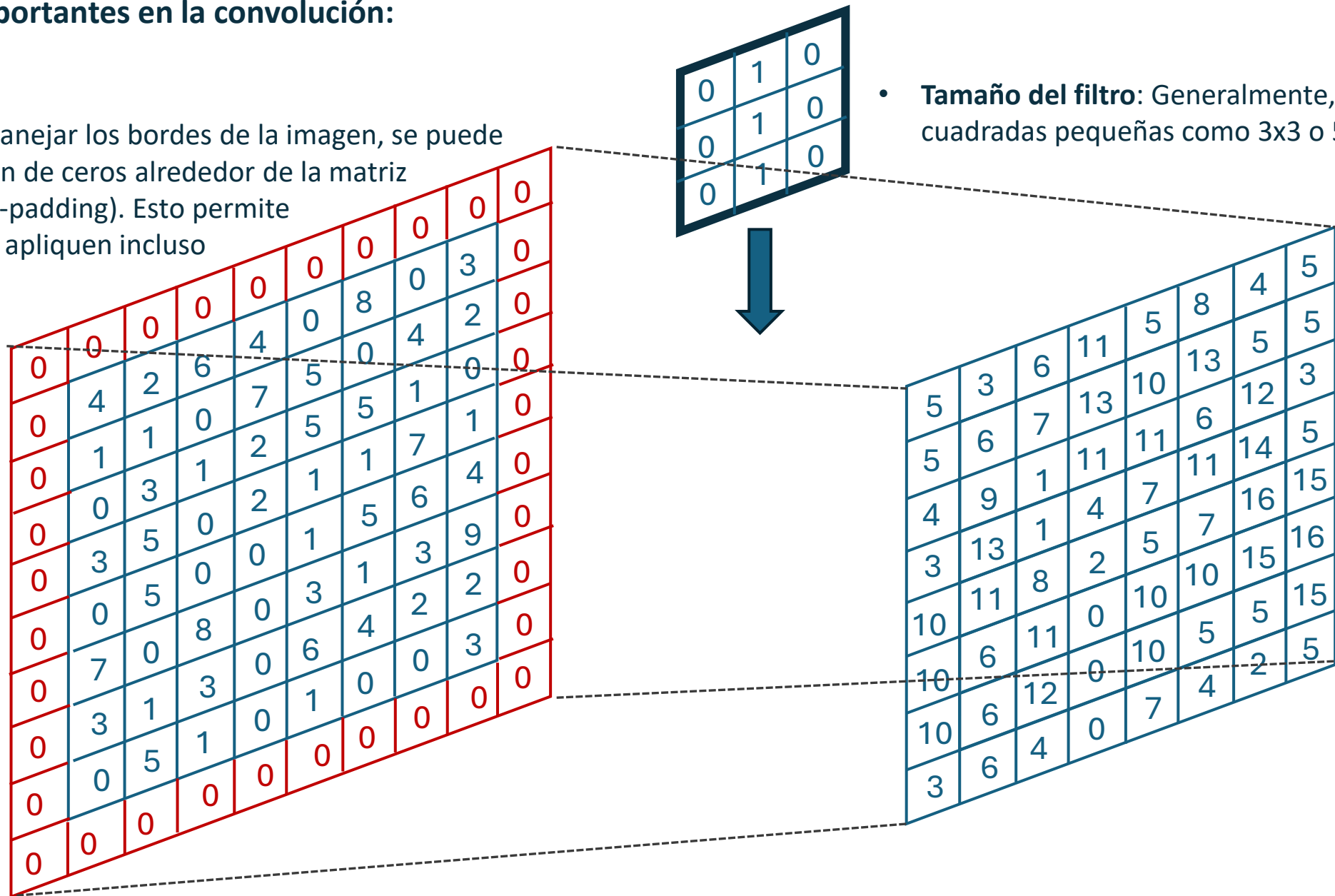


Convolución en el contexto de las CNN



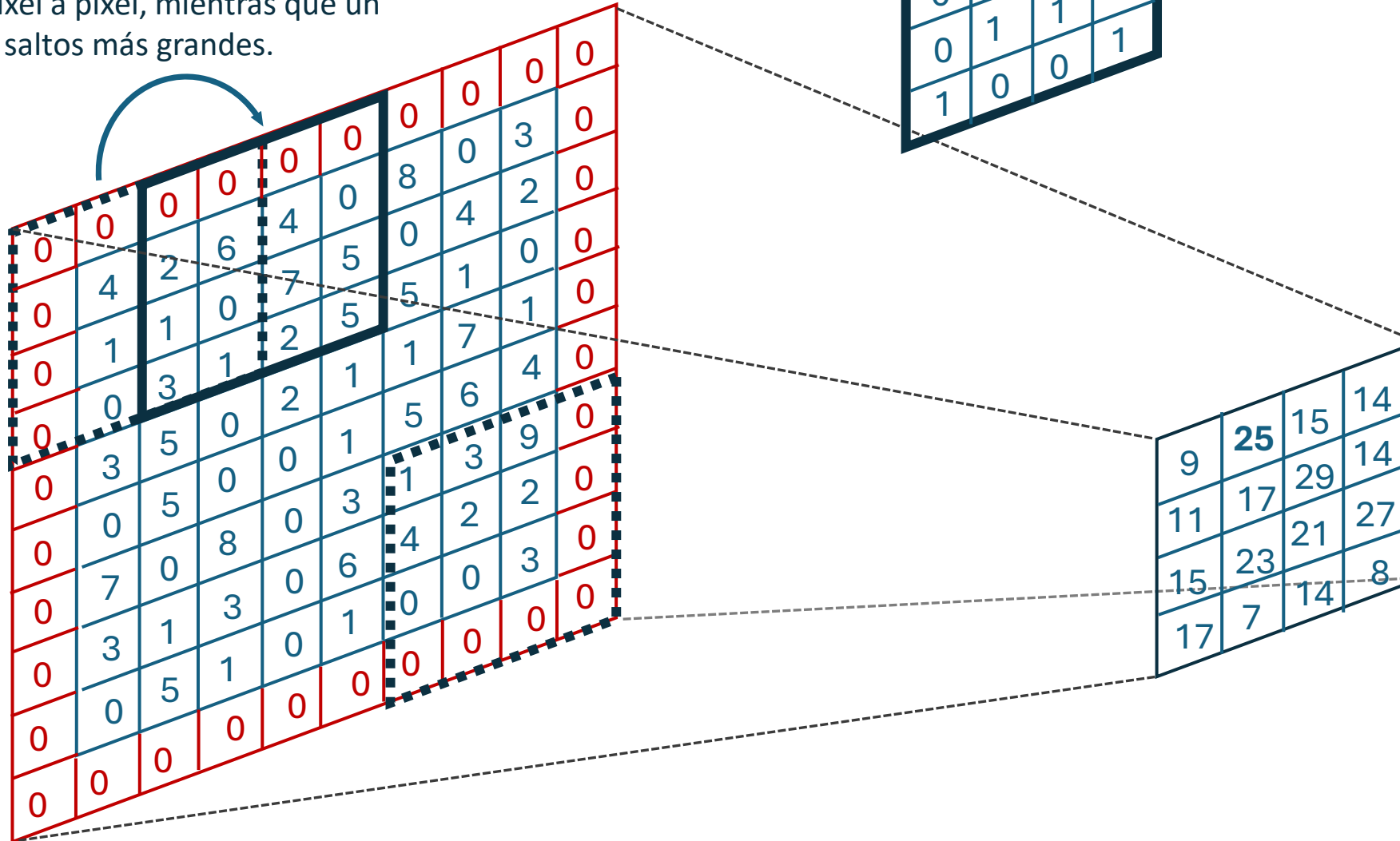
Parámetros importantes en la convolución:

- Padding:** Para manejar los bordes de la imagen, se puede añadir un margen de ceros alrededor de la matriz de entrada (zero-padding). Esto permite que los filtros se apliquen incluso en las esquinas.

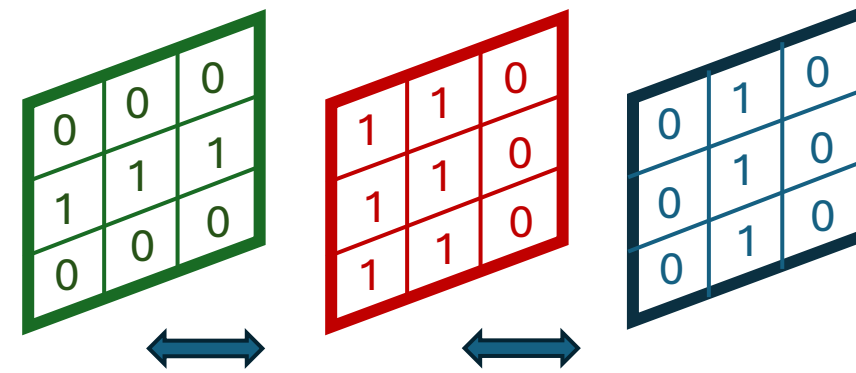
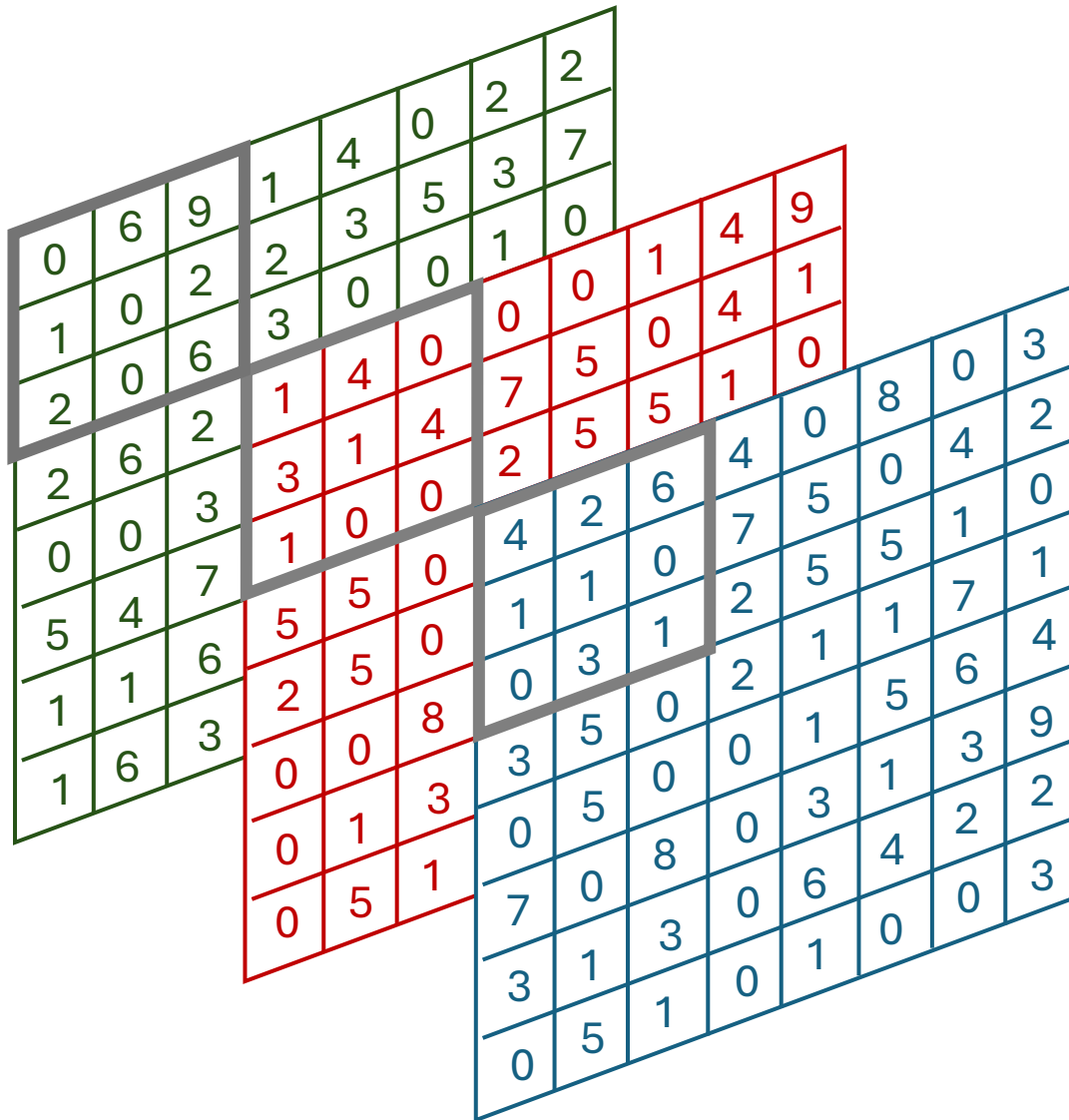


Parámetros importantes en la convolución:

- **Stride:** Determina el número de píxeles que se desplaza el filtro sobre la imagen. Un stride de 1 implica desplazarse píxel a píxel, mientras que un stride de 2 da saltos más grandes.

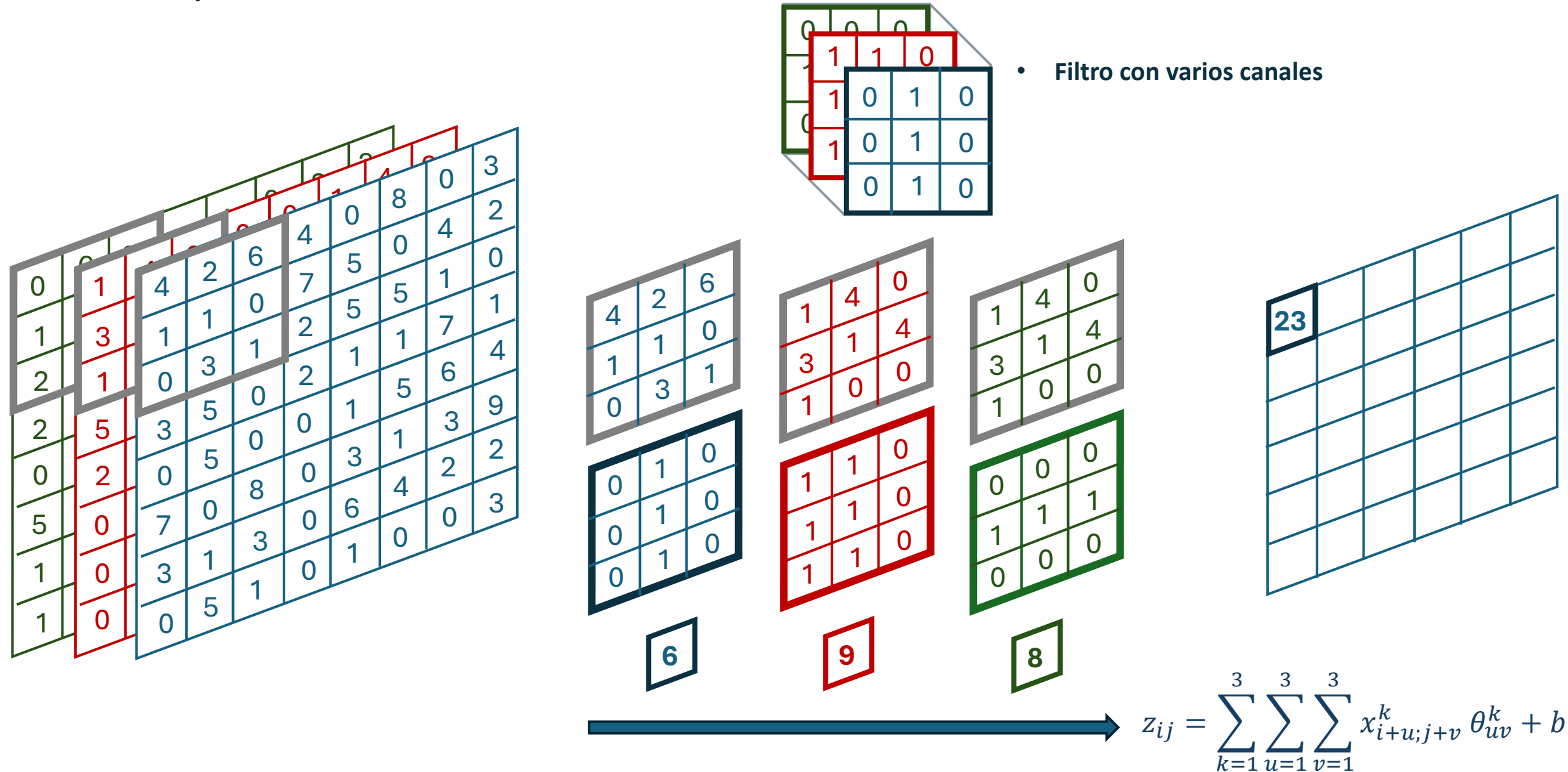


Parámetros importantes en la convolución:



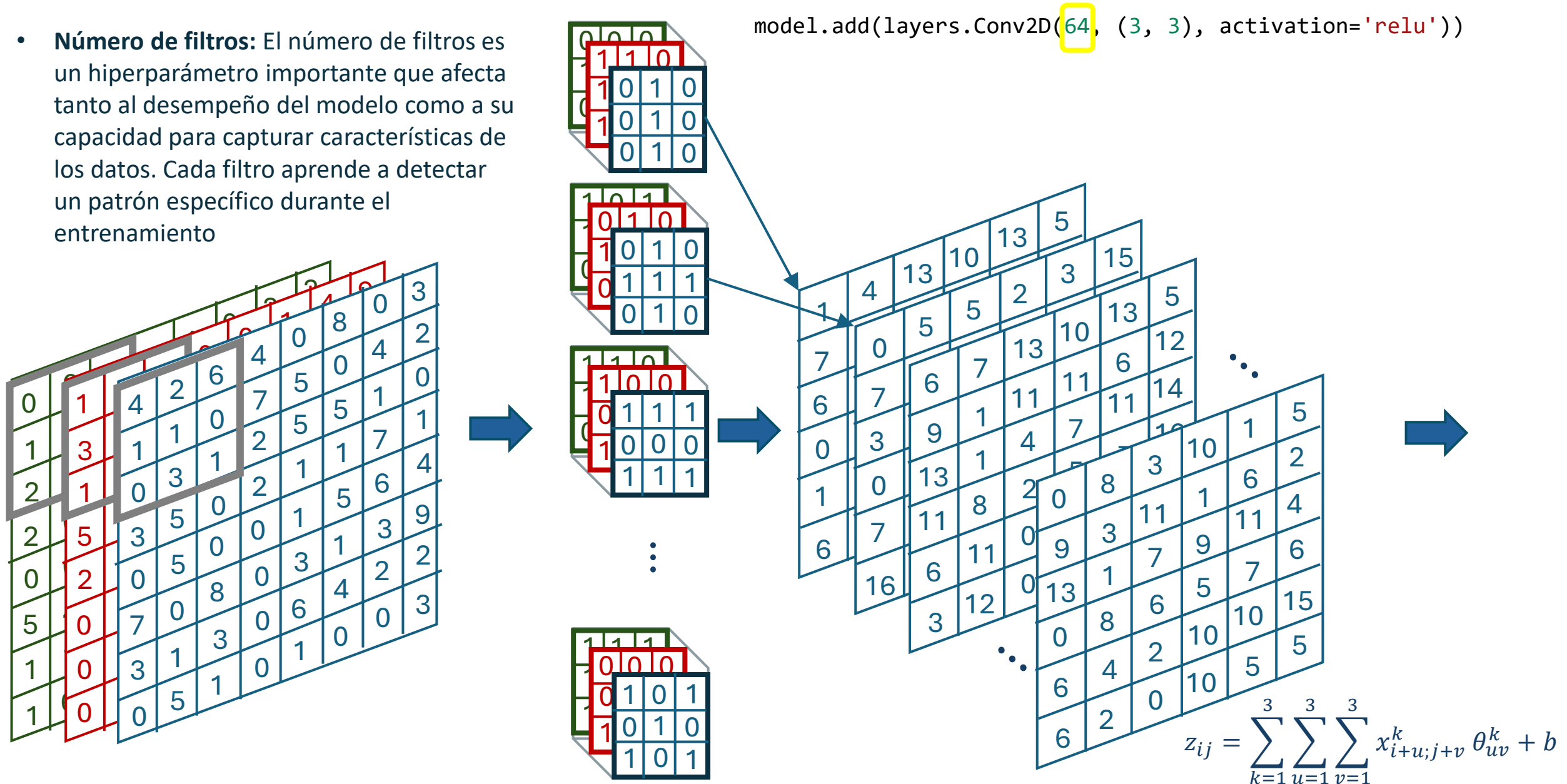
- **Canales:** Para imágenes a color (RGB), el filtro gana en “profundidad” aplicándose de forma simultánea a todos los canales. Esto lo veremos también más adelante cuando apliquemos capas de convolución a la salida de capas intermedias (tensores)

Parámetros importantes en la convolución:



Parámetros importantes en la convolución:

- **Número de filtros:** El número de filtros es un hiperparámetro importante que afecta tanto al desempeño del modelo como a su capacidad para capturar características de los datos. Cada filtro aprende a detectar un patrón específico durante el entrenamiento

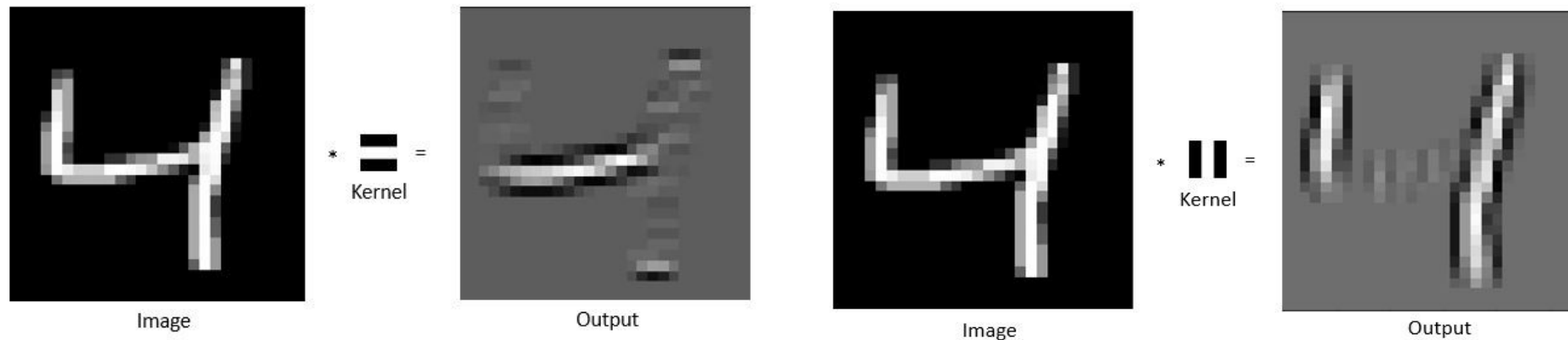


Convolución en el contexto de las CNN

La pieza clave en las capas de convolución son los filtros. Un filtro es una matriz de pesos (kernel) que se desplaza sobre la entrada (imagen, mapa de características, etc.) durante la operación de convolución. **Los valores de cada filtro son los pesos que hay que optimizar durante el entrenamiento.**

Cada filtro aprende a detectar un patrón específico durante el entrenamiento.

- Capas iniciales: Detectan características básicas, como bordes (horizontales, verticales, diagonales), esquinas o colores simples. Los filtros suelen ser pequeños (3×3 o 5×5) para identificar estas características.
- Capas intermedias: Detectan características más complejas, como texturas, formas geométricas, o pequeños detalles de objetos.
- Capas finales: Detectan características globales y abstractas, como la presencia de un ojo en una cara o la forma general de un objeto.



From [pylessons](#)

Convolución en el contexto de las CNN

Ventajas de usar convoluciones:

- **Reducción de dimensionalidad:** La convolución reduce la cantidad de información al concentrarse en las características más importantes. Cada vez que aplicamos un filtro el tamaño del mismo y el *stride* escogido provocan que la salida tenga una menor dimensión que la matriz de características iniciales
- **Invariancia espacial:** Permite que la red reconozca características independientemente de su posición en la imagen.
- **Menor cantidad de parámetros:** Un filtro se aplica a toda la imagen, lo que reduce significativamente el número de parámetros en comparación con las capas completamente conectadas. Más filtros implican más parámetros y, por tanto, mayor costo computacional.
 - El número de filtros debe equilibrarse con los recursos disponibles y la posibilidad de overfitting.
- Demasiados filtros: Aumentan los parámetros del modelo, lo que puede llevar a overfitting, especialmente si el conjunto de datos es pequeño. Incrementan el costo computacional y el tiempo de entrenamiento.
- Muy pocos filtros: El modelo puede no ser capaz de capturar todas las características relevantes, limitando su capacidad para generalizar.

Capas en una Red Neuronal Convolutional.

- **Capas de Pooling:** Su principal función es reducir las dimensiones espaciales de las características extraídas por las capas convolucionales, preservando la información relevante.

4	2	6	4	1	8	0	3
1	1	0	7	5	2	4	2
0	4	0	2	5	5	1	0
3	5	0	2	1	1	7	0
0	5	0	0	1	5	6	6
7	0	8	0	3	1	3	9
2	1	3	0	6	4	3	2
0	5	1	0	1	1	0	3

4	7	8	4
5	2	5	0
7	1	1	9
5	3	6	3

Max pooling: Selecciona el valor máximo de una región específica del mapa de características.

- Conserva las características más importantes, útil para capturar bordes, texturas, o regiones clave de una imagen.
- Reduce el tamaño espacial de los datos de manera eficiente, ayudando a evitar el sobreajuste.

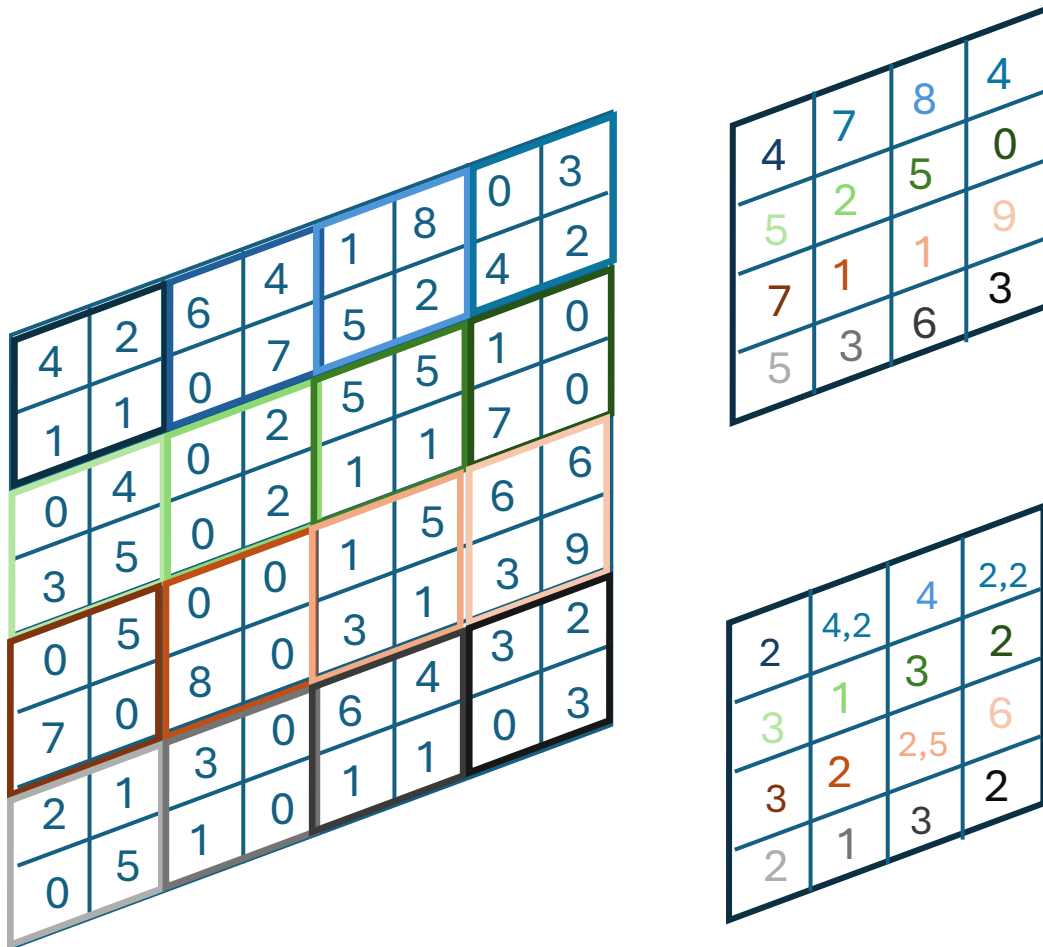
2	4,2	4	2,2
3	1	3	2
3	2	2,5	6
2	1	3	2

Average pooling: Calcula el promedio de los valores dentro de una región específica.

- Suaviza las características, lo que puede ser útil en aplicaciones donde los detalles finos no son tan importantes.
- Asegura que la salida conserve una representación global más uniforme de la información.

Capas en una Red Neuronal Convolutional.

- **Capas de Pooling:** Su principal función es reducir las dimensiones espaciales de las características extraídas por las capas convolucionales, preservando la información relevante.



Hiperparámetros de pooling:

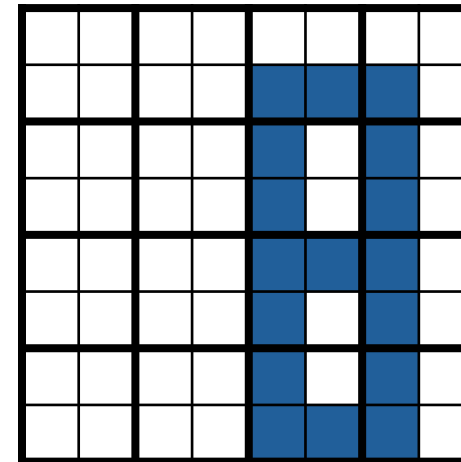
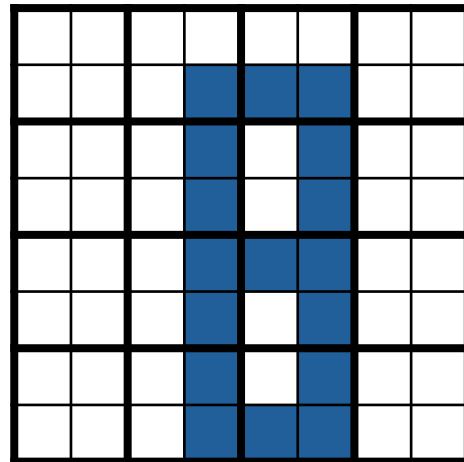
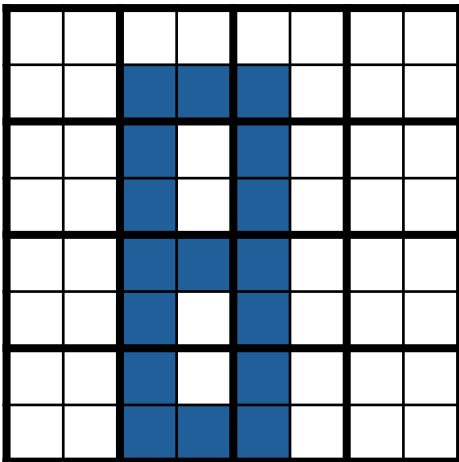
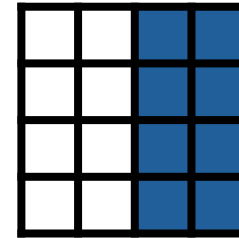
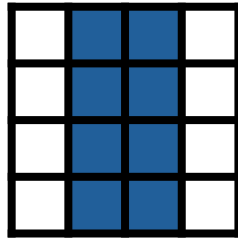
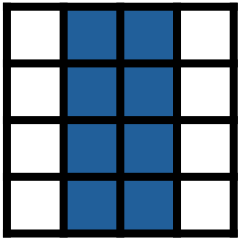
Al aplicar una capa de *pooling*, es importante configurar los siguientes hiperparámetros:

- **Tamaño del *kernel* (filtro):** Determina el tamaño de la región sobre la que se aplica el *pooling*. Un tamaño común es 2x2, pero puede variar según las dimensiones de los datos.
- ***Stride*:** Define cuánto se desplaza el filtro en cada paso. Por ejemplo, un *stride* de 2 con un filtro 2x2 reducirá las dimensiones a la mitad.
- ***Padding*:** Controla si se añaden ceros alrededor del mapa de características antes de aplicar *pooling*. Generalmente, no se usa en *pooling* porque su objetivo principal es reducir dimensiones.

Capas en una Red Neuronal Convolutional.

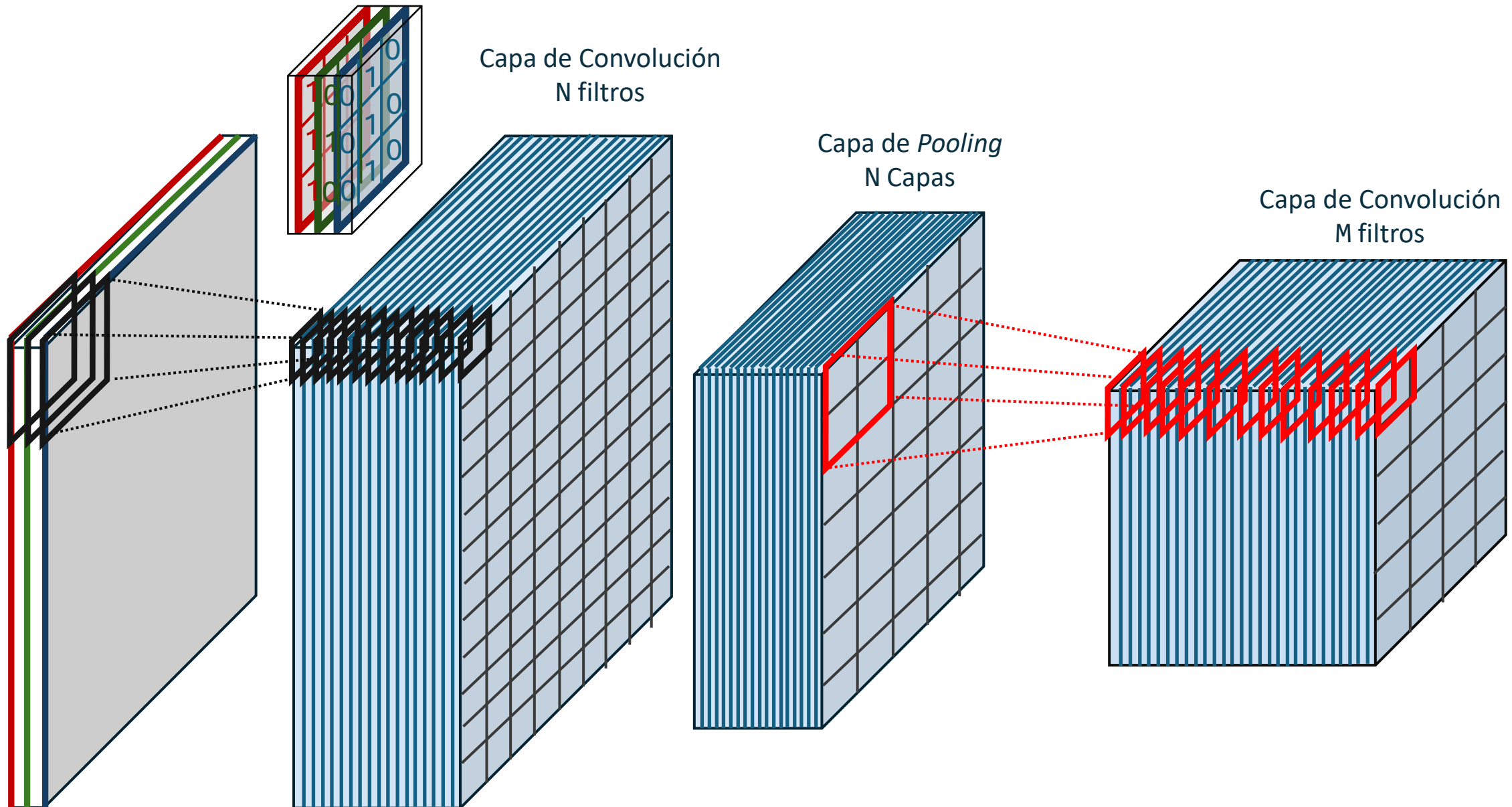
- **Capas de Pooling:** Su principal función es reducir las dimensiones espaciales de las características extraídas por las capas convolucionales, preservando la información relevante.

```
model.add(layers.MaxPooling2D((2, 2)))
```

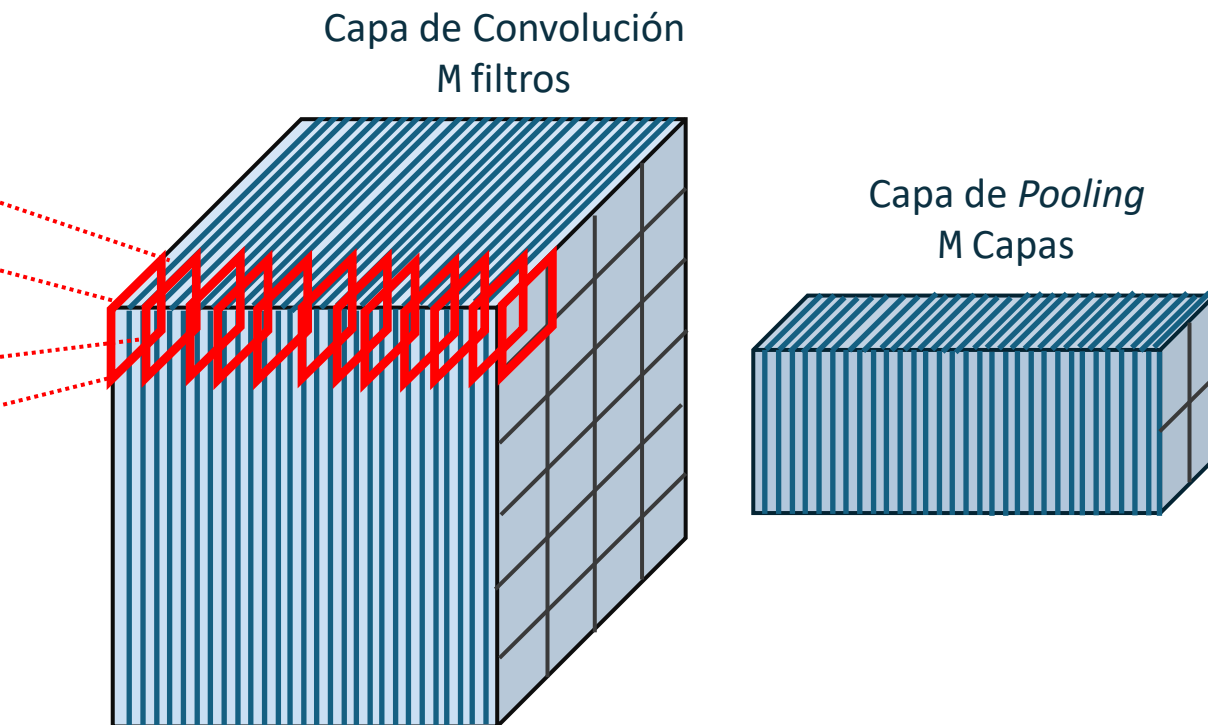


- Introduce invariancia a pequeñas translaciones en la entrada, es decir, si un borde o textura se desplaza ligeramente, seguirá capturándose.

Esquema completo de una CNN

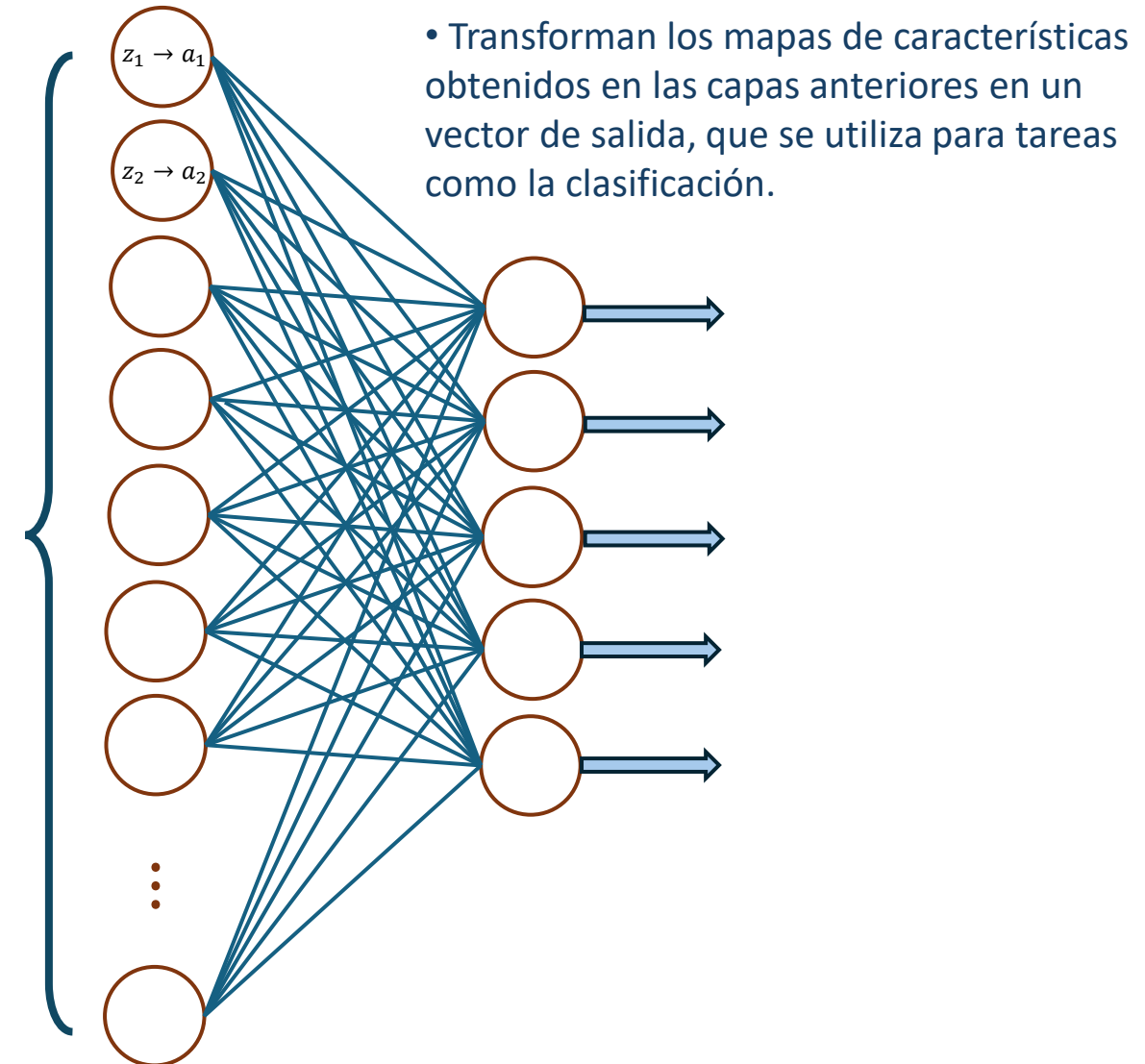


Esquema completo de una CNN

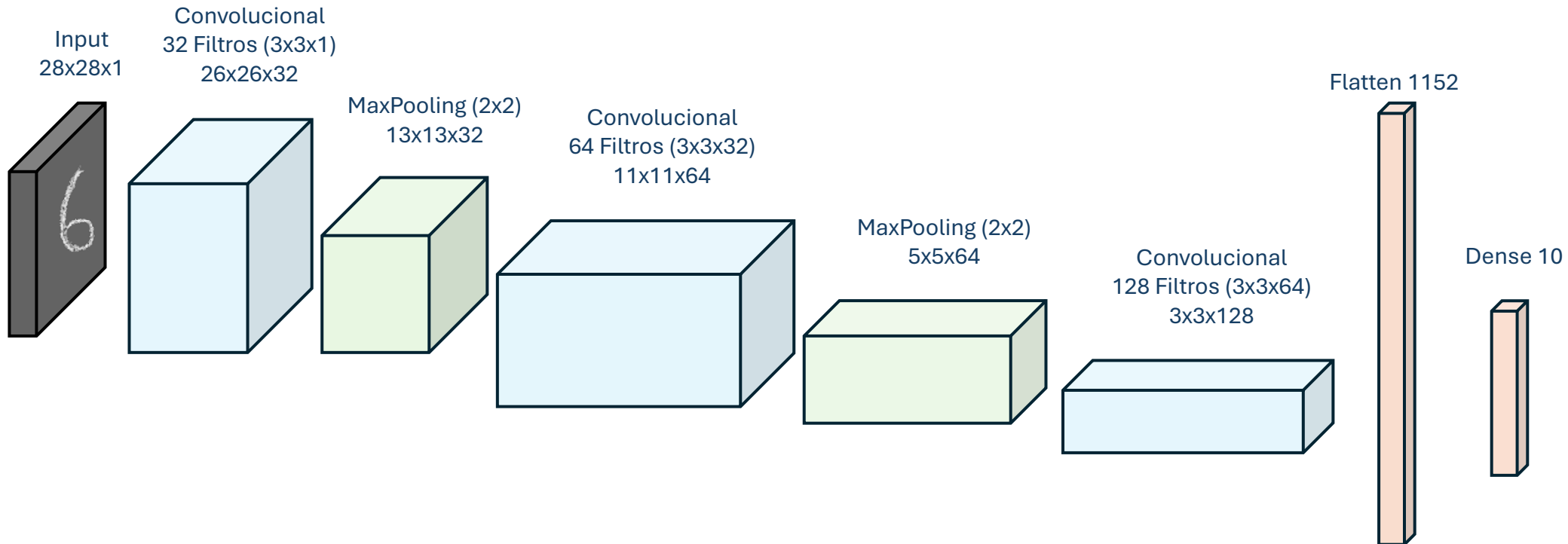


Capas completamente conectadas Dense

- Se colocan al final de la red y se conectan todas las neuronas entre sí.



Ejemplo: MNIST Convolutional Neural Network

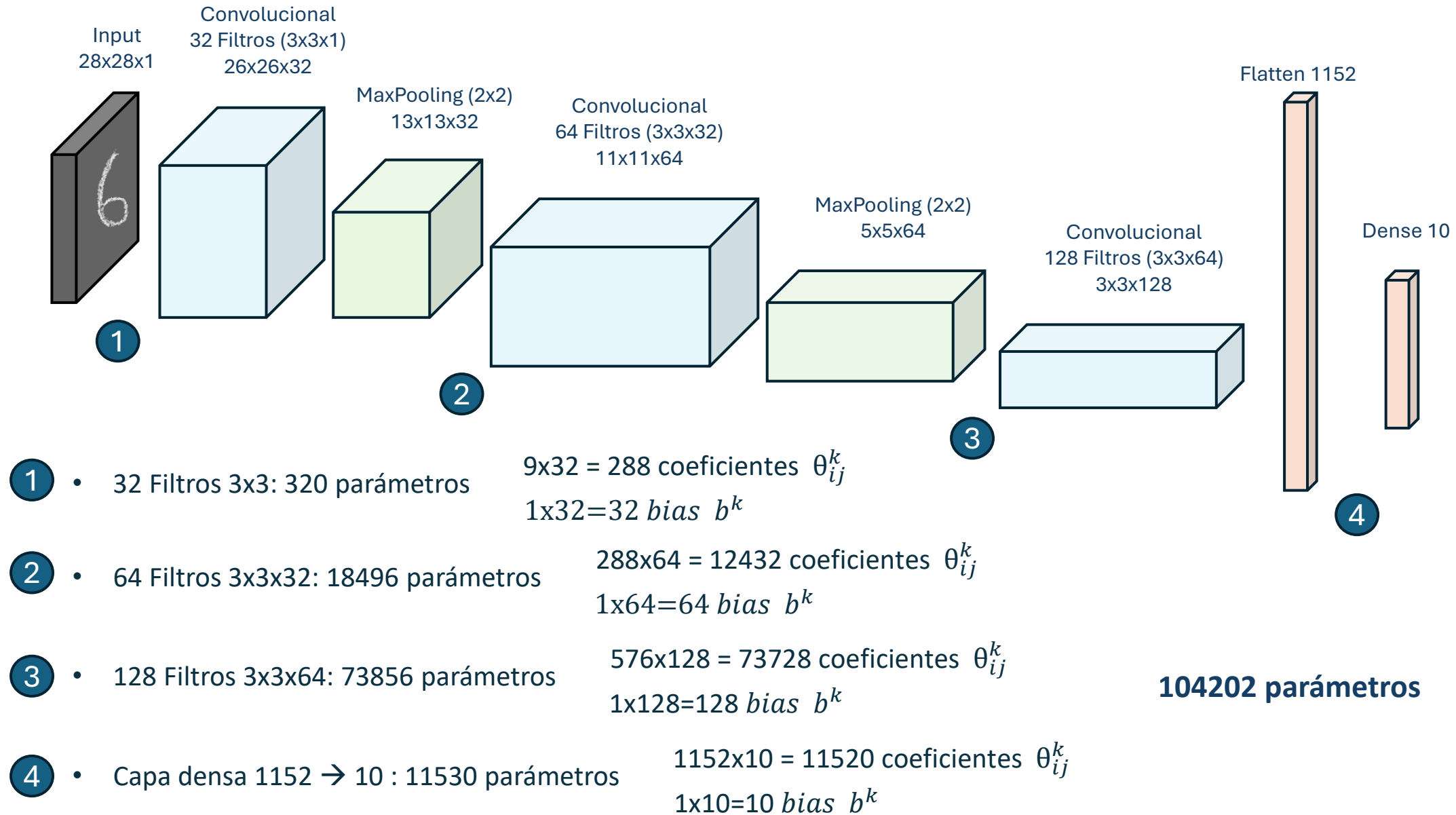


```

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(10, activation='softmax'))

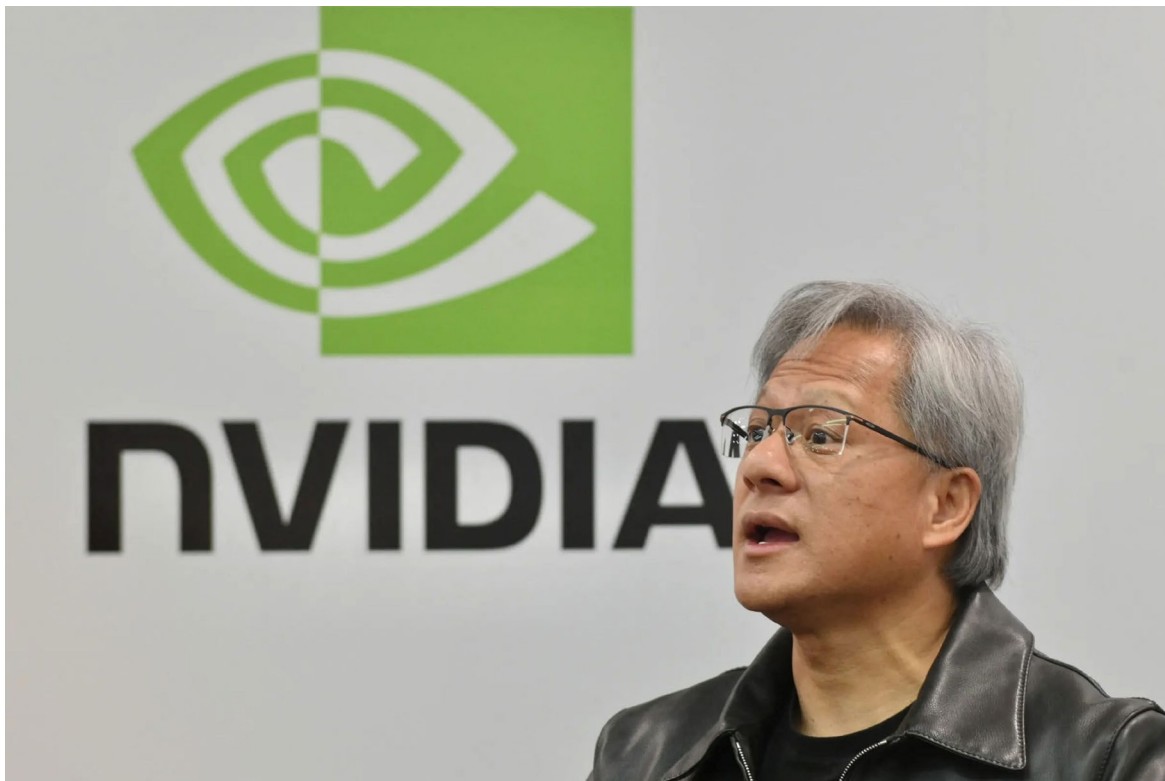
```

Ejemplo: MNIST Convolutional Neural Network



GPUS

Ignacio Gómez
2025



Resumen de mercados > Nvidia

117,26 USD

+110,61 (1.663,31 %) ↑ últimos 5 años

5 mar, 13:57 GMT-5 • Renuncia de responsabilidad

+ Seguir

1 D | 5 D | 1 M | 6 M | YTD | 1 A | 5 A | Máx.



GPUs

Una **GPU (Graphics Processing Unit, Unidad de Procesamiento Gráfico)** es un procesador especializado diseñado originalmente para manejar operaciones gráficas intensivas, como renderizar imágenes, videos y juegos. Sin embargo, debido a su arquitectura altamente paralela, las GPUs también son extremadamente eficientes para realizar cálculos numéricos masivos, lo que las hace ideales para tareas como el entrenamiento de redes neuronales y otros trabajos de computación científica.

Características de una GPU

Compute Performance

•CUDA Cores / Stream Processors:

More cores generally mean the GPU can handle more operations in parallel.

•Tensor Cores (for NVIDIA GPUs):

Tensor cores are specialized for deep learning operations, particularly for matrix multiplications used in neural networks.

Memory Capacity and Bandwidth

•VRAM (Video Memory) Capacity:

A larger memory capacity allows you to train larger models or use bigger batch sizes.

•Memory Bandwidth:

Higher memory bandwidth ensures that data can be read from and written to the GPU memory quickly.

Precision and Throughput

•Floating Point Precision:

GPUs often support multiple precisions (FP32, FP16, INT8, etc.). Modern training routines can leverage mixed-precision (combining FP16 with FP32).

•Specialized Hardware Acceleration:

Some GPUs include dedicated hardware (like NVIDIA's TensorRT) that accelerates inferencing and training tasks.

Software Ecosystem and Compatibility

•Driver and Library Support:

Compatibility with frameworks such as TensorFlow, PyTorch, and CUDA libraries is essential.

•Scalability:

For large-scale training, support for multi-GPU setups (using technologies like NVIDIA NVLink) can help distribute the workload across several GPUs, reducing training time.

GPUs NVIDIA

GPU	Arquitectura	Núcleos CUDA	Núcleos Tensor	Memoria	Ancho de Banda	Precio Aproximado (USD)
A100	Ampere	6,912	432	40/80 GB HBM2	~1.6 TB/s	~\$10,000 – \$15,000
V100	Volta	5,120	640	16/32 GB HBM2	~900 GB/s	~\$8,000 – \$10,000
H100	Hopper	6,144	Nuevos y optimizados	80 GB HBM3	~3 TB/s	~\$25,000 – \$30,000
RTX 3090	Ampere	10,496	~328	24 GB GDDR6X	~936 GB/s	~\$1,500 – \$2,000
RTX 3080	Ampere	8,704	~272	10/12 GB GDDR6X	~760 GB/s	~\$700 – \$900

GPUs NVIDIA

GPU	Potencia consumida (W)
A100	~250W (versión PCIe) / ~400W (versión SXM)
V100	~250W (versión PCIe) / ~300W (versión SXM2)
H100	~700W
RTX 3090	~350W
RTX 3080	~320W



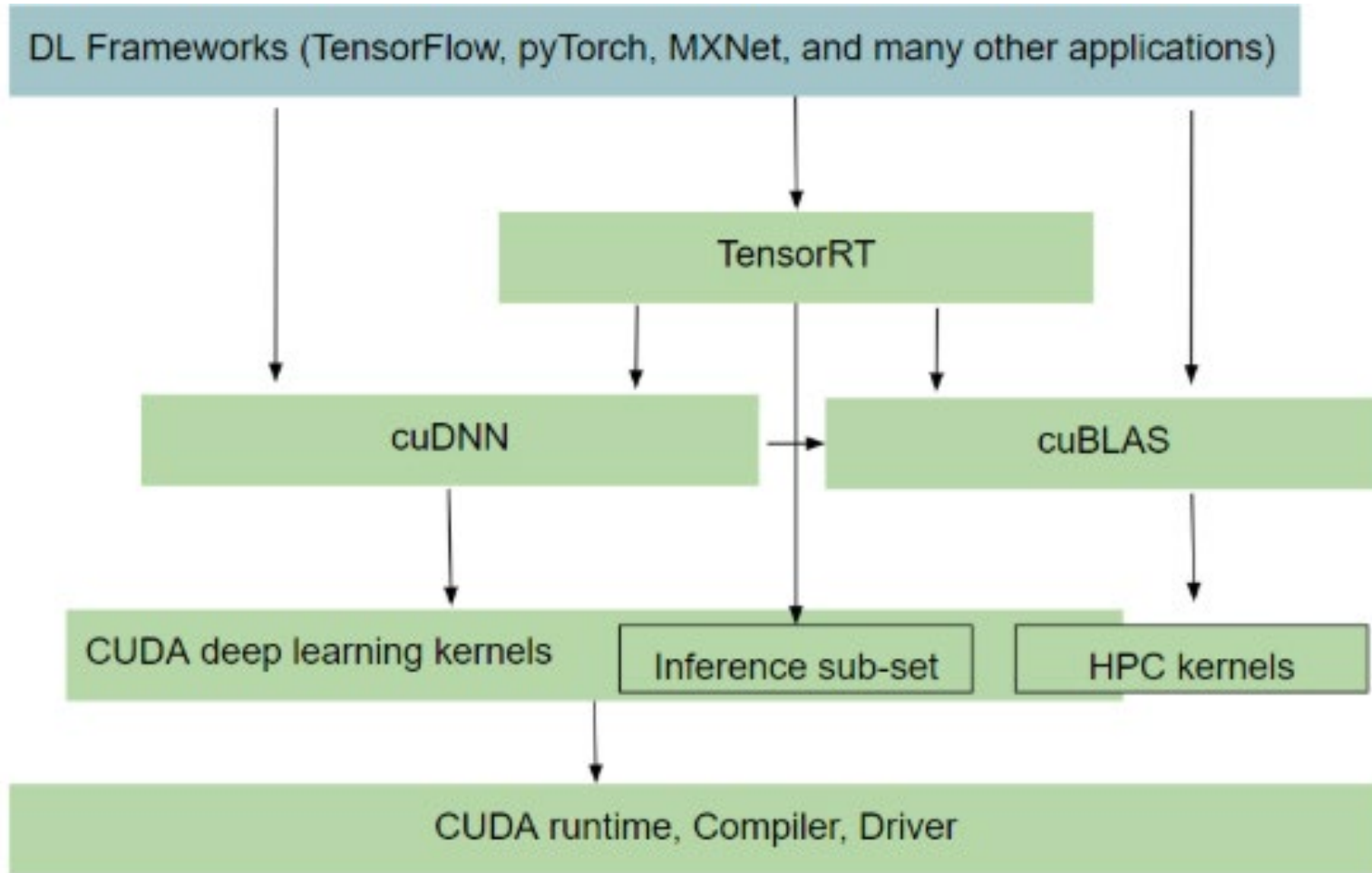
Max Star Estufa Electrica Bajo Consumo de Cuarzo Calefactor de Baño Infrarrojo 3 Posiciones de Calor 400W/800/1200W Asa de Transporte...

Por Qué Nos Encanta

- 🔥 Potente y rápido calentamiento.
- 🔥 Personaliza la temperatura fácilmente.
- 🔥 Operación completamente silenciosa.

[Mostrar Más](#)

Librerias NVIDIA



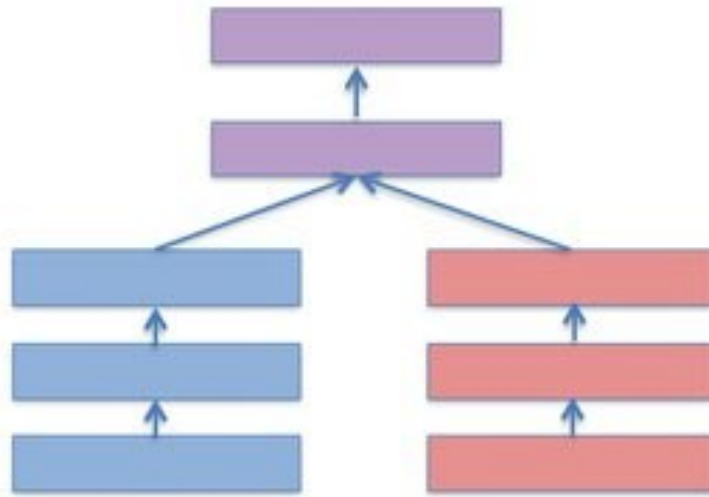
ARQUITECTURAS MAS COMPLEJAS

KERAS FUNCTIONAL API

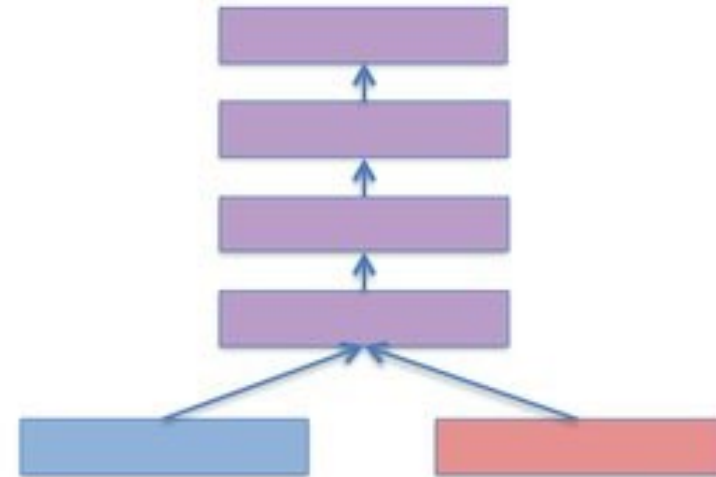
Ignacio Gómez
2025

Juntar dos clasificadores

Si queremos datos estructurados al clasificador no tiene sentido hacerlos pasar por la red convolucional, lo lógico es añadirlos directamente a la etapa final de clasificación:



Late fusion, "red" and "blue" features are in different domains.
Ex: Image+text or image+metadata



Early fusion, "red" and "blue" features are similar domains.
Ex: different channels in color+depth images

Keras functional API

Keras ofrece dos formas principales de crear modelos: el **Modelo Secuencial (Sequential Model)** y la **API Funcional**.

La **API Funcional de Keras** es una forma más flexible y avanzada de crear modelos de redes neuronales en comparación con el Modelo Secuencial. Mientras que el Modelo Secuencial es adecuado para pilas simples de capas donde cada capa tiene exactamente un tensor de entrada y un tensor de salida, la API Funcional permite crear modelos más complejos con múltiples entradas, múltiples salidas, conexiones no lineales y compartimiento de capas.

Características principales de la API Funcional:

- 1.Modelos no lineales:** Puedes crear modelos con ramificaciones, múltiples entradas y salidas, y conexiones complejas entre capas.
- 2.Reutilización de capas:** Puedes reutilizar las mismas capas en diferentes partes del modelo.
- 3.Modelos más complejos:** Es ideal para arquitecturas avanzadas como redes residuales, redes multitarea, modelos con atención, etc.

En esta forma de construir el modelo, la red se diseña como si construyéramos un grafo acíclico.

Keras functional API

```
from keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense
from keras.models import Model

# Definir la entrada (imágenes de 28x28 píxeles con 1 canal, como en MNIST)
input_layer = Input(shape=(28, 28, 1))

# Primera capa convolucional
conv1 = Conv2D(filters=32, kernel_size=(3, 3), activation='relu')
X = conv1(input_layer)
x = MaxPooling2D(pool_size=(2, 2))(x)

# Segunda capa convolucional
x = Conv2D(filters=64, kernel_size=(3, 3), activation='relu')(x)
x = MaxPooling2D(pool_size=(2, 2))(x)

# Aplanar la salida para conectarla a una capa densa
x = Flatten()(x)

# Capa densa (fully connected)
x = Dense(units=128, activation='relu')(x)

# Capa de salida (por ejemplo, para 10 clases en MNIST)
output_layer = Dense(units=10, activation='softmax')(x)

# Crear el modelo
model = Model(inputs=input_layer, outputs=output_layer)
```

Juntar dos clasificadores

```
from keras.models import Model
from keras.layers import Input, Dense, concatenate, Conv2D, MaxPooling2D, Flatten, Dropout
from keras.utils import plot_model

imagen_input=Input(shape=(32, 32, 3), name='Imagenes')
conv1 = Conv2D(64, (3, 3), activation='relu')(imagen_input)
max1 = MaxPooling2D((2, 2))(conv1)
drop1 = Dropout(0.25)(max1)
conv2 = Conv2D(128, (3, 3), activation='relu')(drop1)
max2 = MaxPooling2D((2, 2))(conv2)
drop2= Dropout(0.25)(max2)
conv3 = Conv2D(256, (3, 3), activation='relu')(max2)
max3 = MaxPooling2D((2, 2))(conv3)
drop3 = Dropout(0.25)(max3)
imagen_output = Flatten()(drop3)

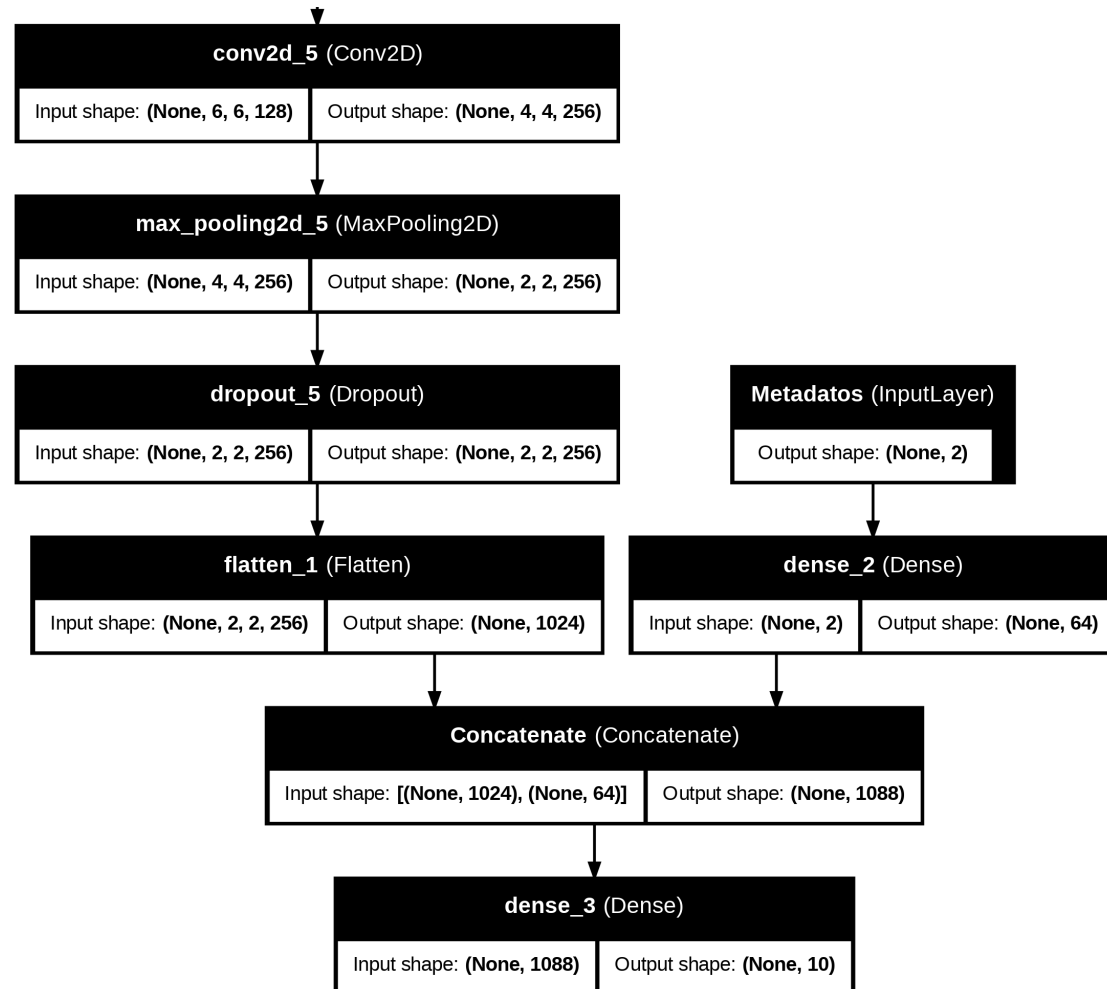
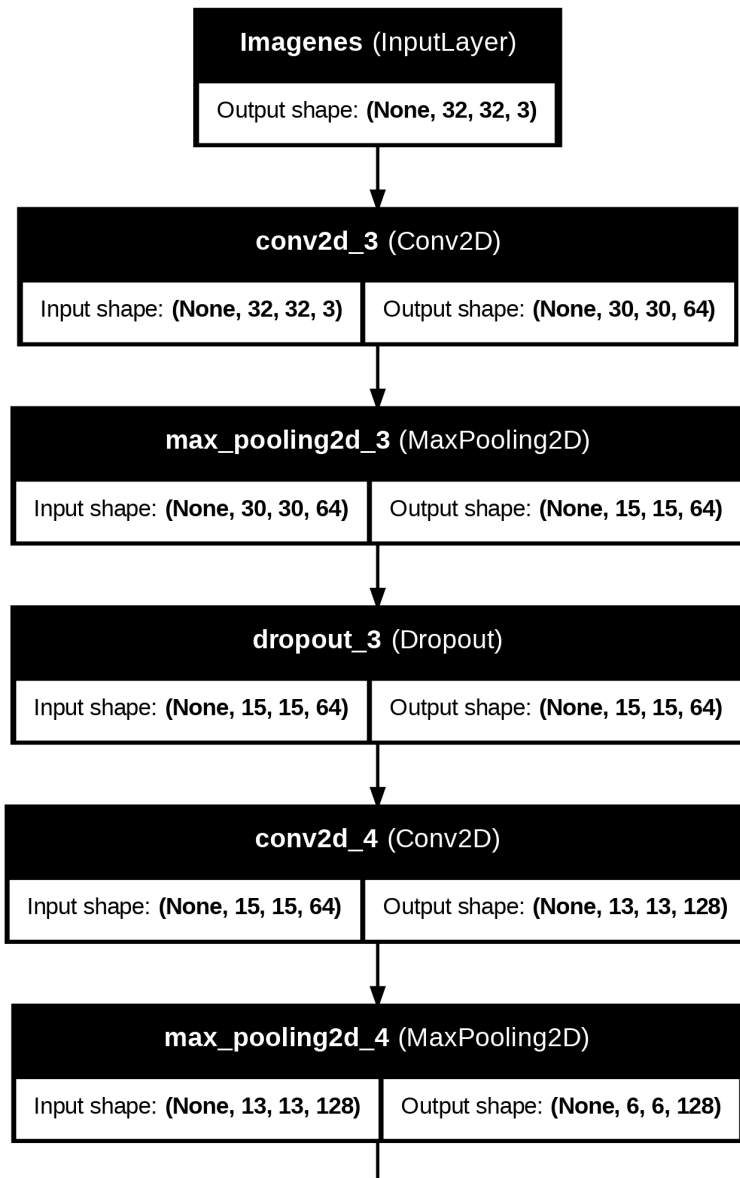
metadatos_input=Input(shape=(2,), name='Metadatos')
meta_output = Dense(64, activation='relu')(metadatos_input)

concat = concatenate([imagen_output, meta_output], name='Concatenate')
final_model_output = Dense(10, activation='softmax')(concat)

final_model = Model(inputs=[imagen_input, metadatos_input], outputs=final_model_output, name='Final_output')
final_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Dibujar un esquema de la red

```
from keras.utils import plot_model  
plot_model(final_model, to_file='model_plot.png',  
show_shapes=True, show_layer_names=True)
```



CONVOLUCIONES 1D

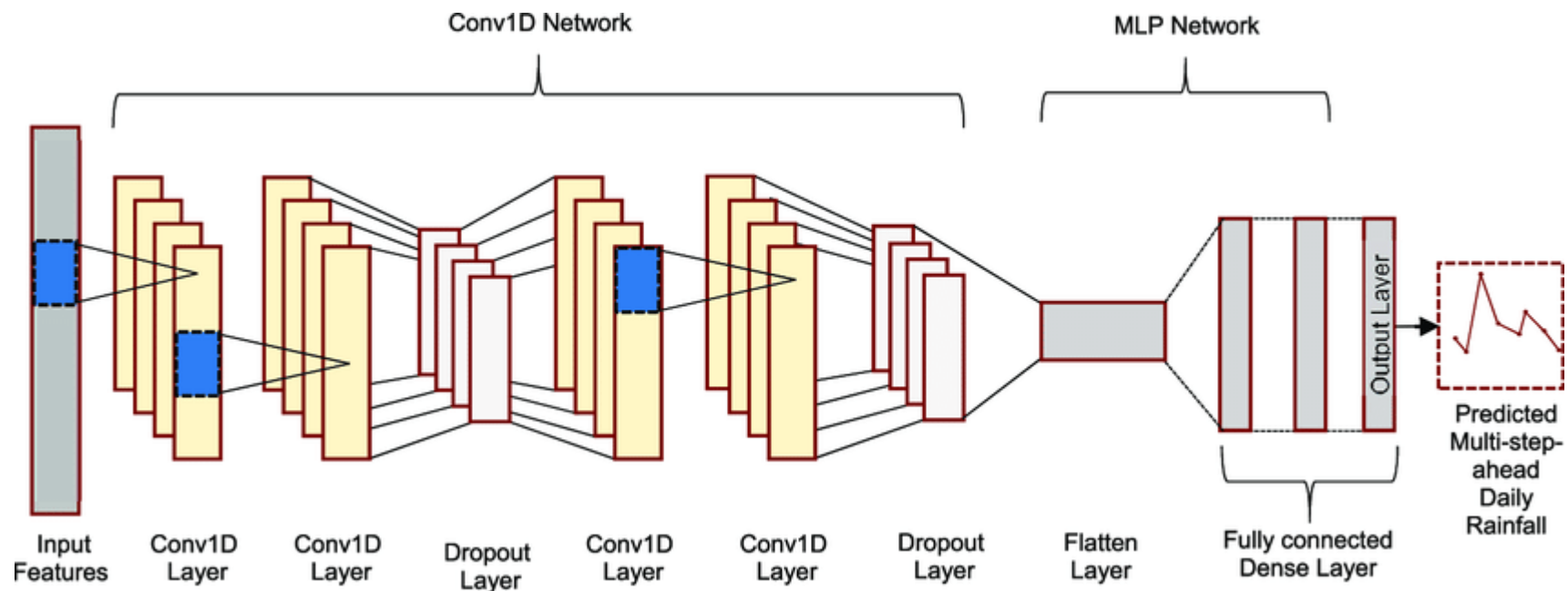
Ignacio Gómez
2025

Convoluciones 1D

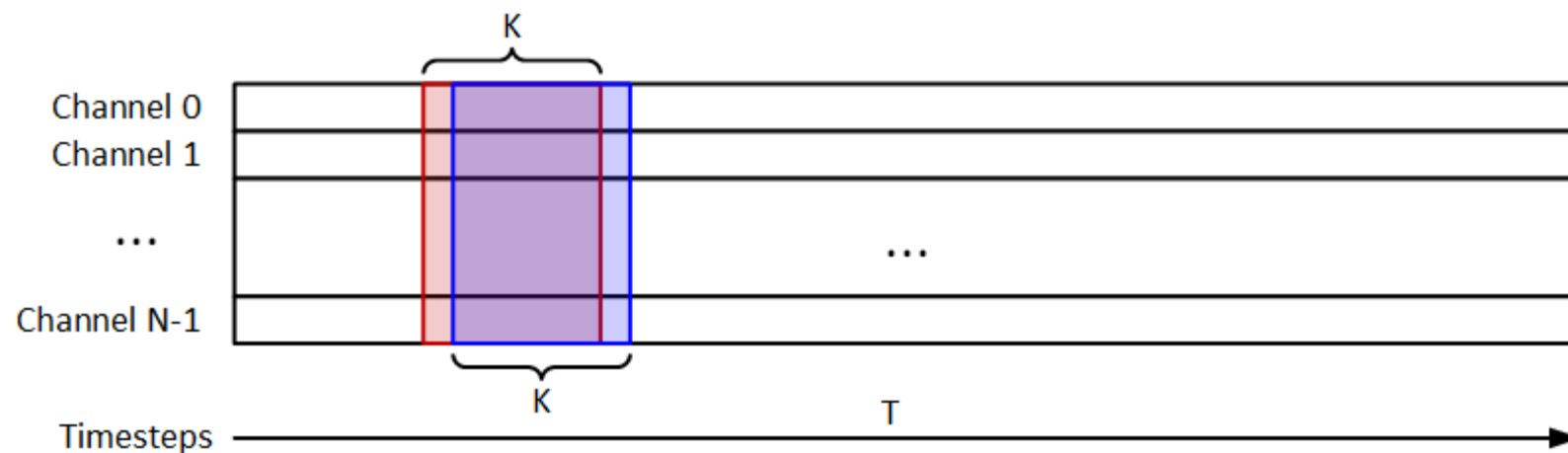
Las capas convolucionales 1D son un tipo de capa utilizada en redes neuronales convolucionales (CNN) que se aplican principalmente a datos secuenciales o unidimensionales, como series temporales, señales de audio, texto (cuando se representa como secuencias de embeddings) o cualquier otro tipo de dato que tenga una estructura secuencial.

¿Para qué sirven?

- 1.Extracción de características locales:** Las convoluciones 1D permiten detectar patrones locales en secuencias, como picos en señales, tendencias en series temporales o estructuras en texto.
- 2.Reducción de dimensionalidad:** Al aplicar filtros (kernels) que se deslizan sobre la secuencia, se reduce la cantidad de parámetros en comparación con capas densas.
- 3.Invariante a traslaciones:** Pueden identificar patrones independientemente de su posición en la secuencia.
- 4.Eficiencia computacional:** Al compartir parámetros (filtros), son más eficientes que las capas densas para datos secuenciales.

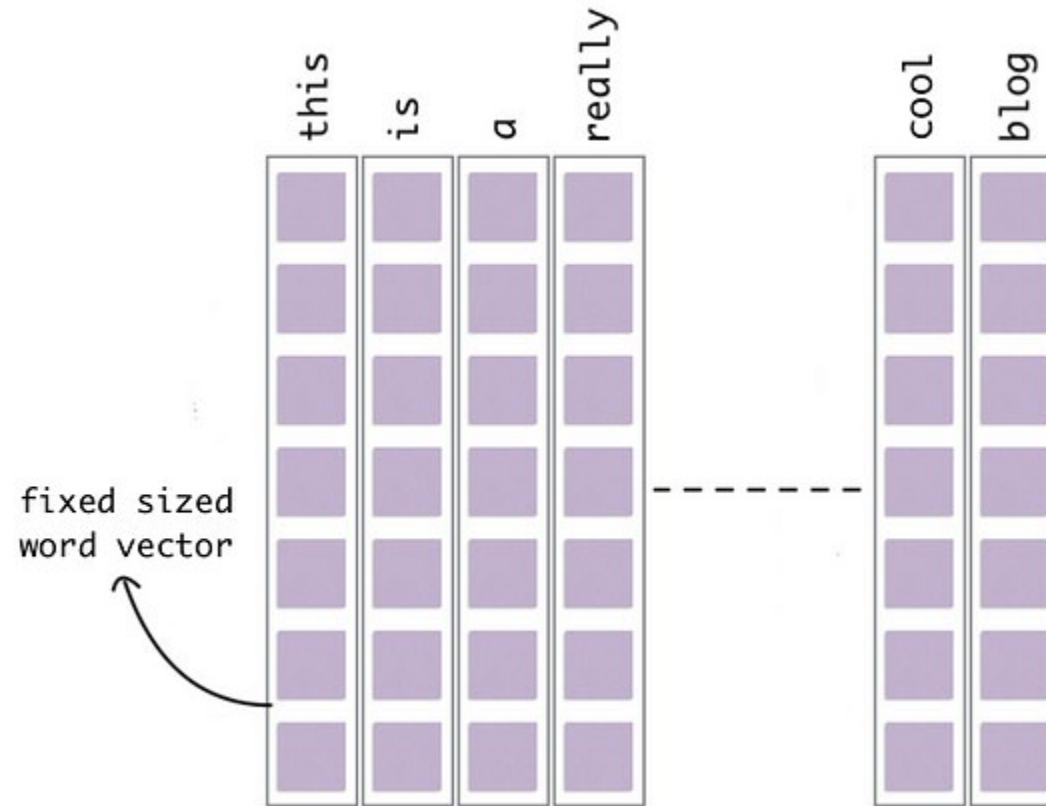


Conv1D (filters= F ,
kernel_size= K , ...)



Convoluciones 1D

Convoluciones 1D para texto



Ejemplo: UCI Human Activity Dataset

Es un conjunto de datos ampliamente utilizado en el campo del aprendizaje automático y la inteligencia artificial para la clasificación y reconocimiento de actividades humanas. Este dataset fue creado por el **Centro de Aprendizaje Automático y Sistemas Inteligentes de la Universidad de California, Irvine (UCI)**, y contiene datos capturados mediante sensores (acelerómetros y giroscopios) integrados en un smartphone y etiquetados:

- Caminar (Walking)
- Subir escaleras (Walking Upstairs)
- Bajar escaleras (Walking Downstairs)
- Sentarse (Sitting)
- Estar de pie (Standing)
- Acostarse (Laying)

•**Datos:** El dataset contiene mediciones de sensores (acelerómetro y giroscopio) en tres ejes (x, y, z) capturados a una frecuencia de 50 Hz (50 muestras por segundo).

•**Participantes:** Los datos fueron recolectados de 30 voluntarios (entre 19 y 48 años) mientras realizaban las actividades mencionadas.

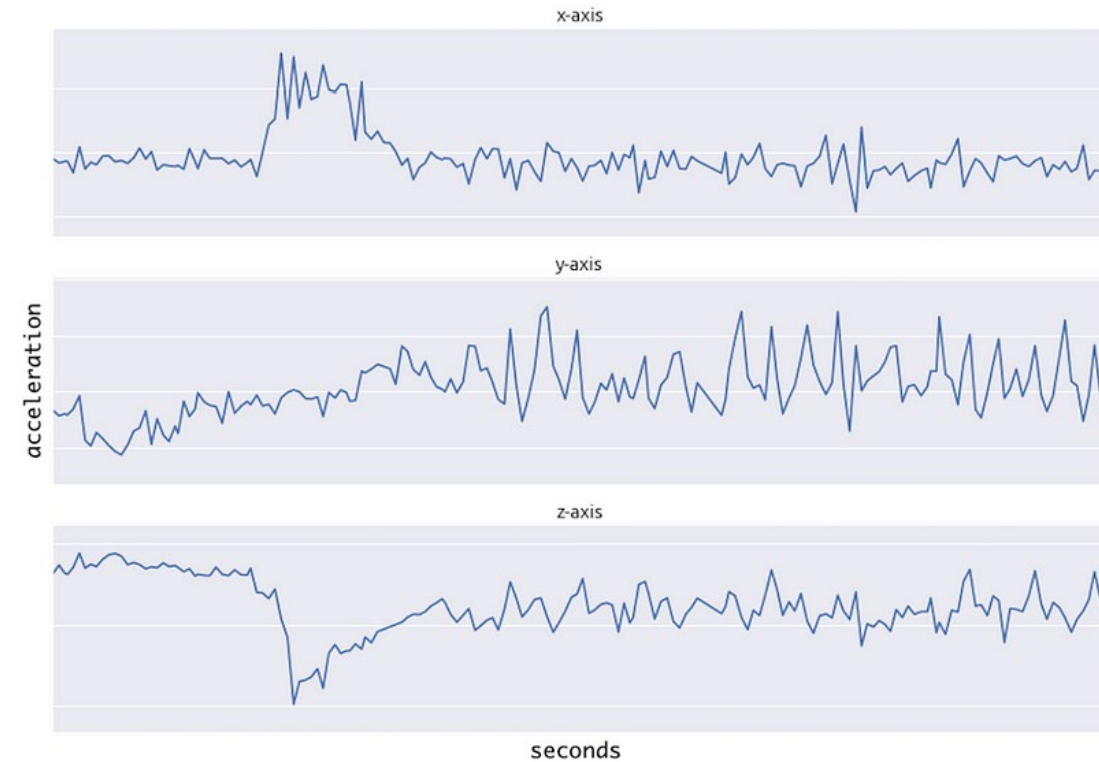
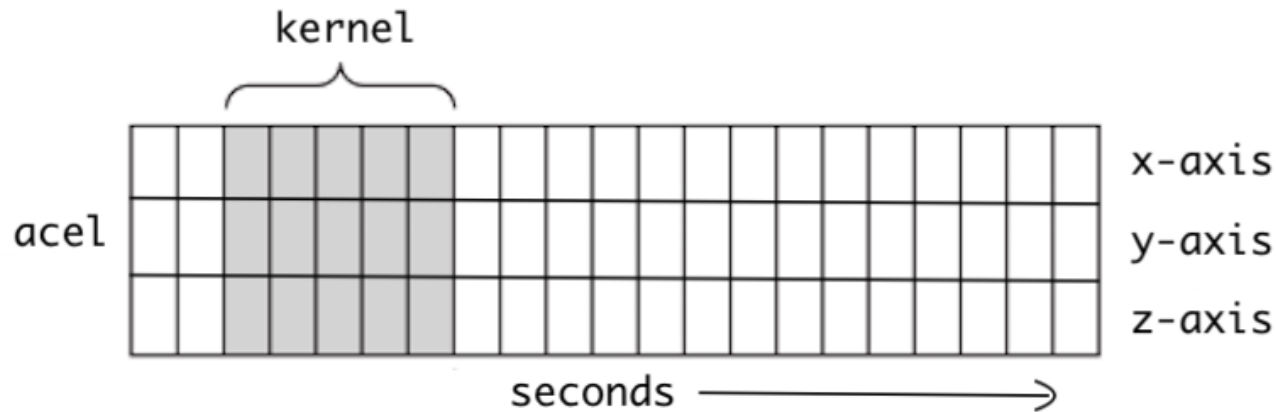
•**Preprocesamiento:** Las señales de los sensores se preprocesaron para eliminar ruido y se dividieron en ventanas de 2.56 segundos con un solapamiento del 50%, lo que resulta en 128 muestras por ventana.

•**Características extraídas:** Para cada ventana, se calcularon diversas características estadísticas (media, desviación estándar, energía, entropía, correlación entre ejes, etc.), lo que resulta en un total de 561 características por ventana.

Ejemplo: UCI Human Activity Dataset

En el artículo original (citado abajo), se extraían más de 500 características (medias, varianzas, coeficientes de Fourier, etc..) y se entrenaba un modelo SVM. Se conseguía una precisión global del 89%.

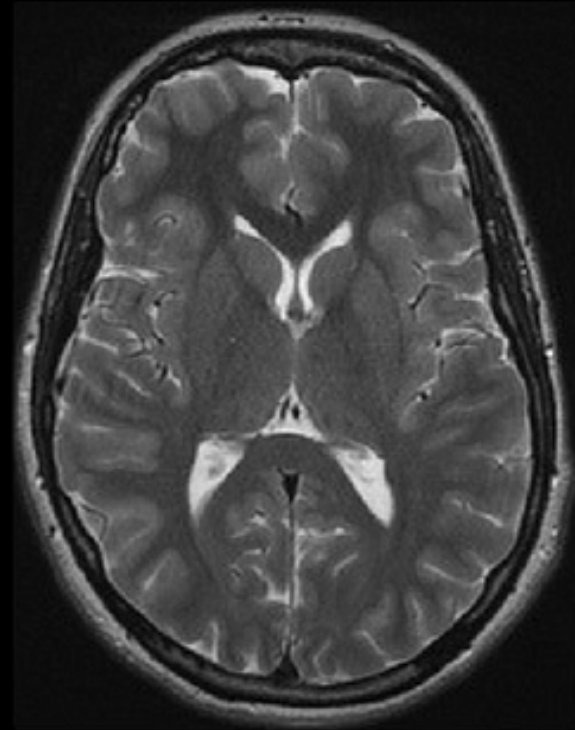
Nosotros vamos a utilizar una red Convolutional con filtros 1D que recorran la escala temporal de las muestras:



[Human Activity Recognition on Smartphones Using a Multiclass Hardware-Friendly Support Vector Machine](#)

Conv3D

CT



MRI

MLFLOW

Ignacio Gómez
2025

MLFlow

MLflow es una plataforma de código abierto diseñada para gestionar el ciclo de vida completo de los proyectos de machine learning (ML). Desarrollada por Databricks, MLflow facilita la organización, seguimiento y despliegue de modelos de machine learning, ayudando a los equipos a trabajar de manera más eficiente y colaborativa. MLflow se compone de cuatro módulos principales:

1.MLflow Tracking: Permite registrar y seguir experimentos de machine learning. Puedes guardar parámetros, métricas, artefactos (como modelos) y visualizar resultados en una interfaz web. Esto facilita la comparación de diferentes enfoques y la reproducibilidad de los experimentos.

2.MLflow Projects: Proporciona un formato estándar para empaquetar código de machine learning, lo que facilita su ejecución en diferentes entornos. Un proyecto MLflow puede incluir dependencias, entornos y scripts, lo que permite una fácil replicación y colaboración.

3.MLflow Models: Ofrece un formato estándar para empaquetar modelos de machine learning, independientemente de la biblioteca o framework utilizado (como TensorFlow, PyTorch, Scikit-learn, etc.). Esto facilita el despliegue de modelos en diferentes entornos, como servicios REST, bases de datos o sistemas de streaming.

4.MLflow Registry: Es un repositorio centralizado para gestionar el ciclo de vida de los modelos. Permite versionar, anotar y transicionar modelos entre etapas (por ejemplo, de "staging" a "production"), lo que facilita la gestión colaborativa y el control de calidad.

MLFlow Tracking

Es un módulo de MLflow diseñado para registrar y organizar experimentos de machine learning. Su objetivo principal es facilitar el seguimiento de los diferentes aspectos de un experimento, como los parámetros, métricas, artefactos y el código utilizado, lo que permite a los científicos de datos comparar, reproducir y analizar los resultados.

- 1.Registro de parámetros:** Permite guardar los hiperparámetros y configuraciones utilizadas en cada ejecución de un experimento (por ejemplo, la tasa de aprendizaje, el número de capas en una red neuronal, etc.).
- 2.Registro de métricas:** Puedes registrar métricas clave (como precisión, pérdida, F1-score, etc.) durante el entrenamiento o evaluación de un modelo.
- 3.Almacenamiento de artefactos:** Permite guardar archivos relacionados con el experimento, como modelos entrenados, gráficos, archivos de configuración o cualquier otro archivo relevante.
- 4.Interfaz de usuario (UI):** MLflow proporciona una interfaz web donde puedes visualizar y comparar los resultados de diferentes ejecuciones de experimentos.
- 5.Reproducibilidad:** MLflow registra automáticamente información sobre el entorno de ejecución, como la versión del código, el entorno de Python y las dependencias utilizadas.
- 6.Integración con múltiples frameworks:** MLflow Tracking es compatible con una amplia variedad de bibliotecas y frameworks de machine learning, como Keras, TensorFlow, PyTorch, Scikit-learn, XGBoost, etc.
- 7.Almacenamiento flexible:** Los datos de seguimiento (parámetros, métricas, artefactos) pueden almacenarse localmente (en el sistema de archivos) o en servidores remotos (como bases de datos o almacenamiento en la nube).

MLFlow Tracking

```
import mlflow

# Inicia una ejecución de experimento
with mlflow.start_run():
    # Registra parámetros
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_param("epochs", 10)

    # Entrenamiento de un modelo

    # Registra métricas
    mlflow.log_metric("loss", loss)
    mlflow.log_metric("accuracy", accuracy)

    # Guarda un artefacto (por ejemplo, un modelo)
    with open("model.txt", "w") as f:
        f.write("Modelo entrenado")
    mlflow.log_artifact("model.txt")
```

MLFlow Tracking con Keras

```
mlflow server --host 127.0.0.1 --port 8080
```

```
import mlflow

mlflow.set_tracking_uri(uri="http://127.0.0.1:8080")

# Create a new MLflow Experiment
mlflow.set_experiment("Ejemplo con MLflow")

run = mlflow.start_run()

history = model.fit(x_train, y_train, epochs=10, batch_size=32, validation_data=(x_test,
y_test), callbacks=[mlflow.keras.MlflowCallback(run)])

mlflow.end_run()
```


MLFlow Tracking User Interface

Runs

Evaluation

Experimental

Traces

Q

metrics.rmse < 1 and params.model = "tree"

Time created

State: Active

Datasets

Sort: Created

Columns

Group by

	Run Name	Created	Dataset	Duration	Source	Models
☐	secretive-vole-694	19 hours ago	-	43.0s	c:\Users\...	-
☐	luxuriant-bird-543	19 hours ago	-	42.7s	c:\Users\...	-
☐	skillful-pug-611	19 hours ago	-	46.3s	c:\Users\...	-

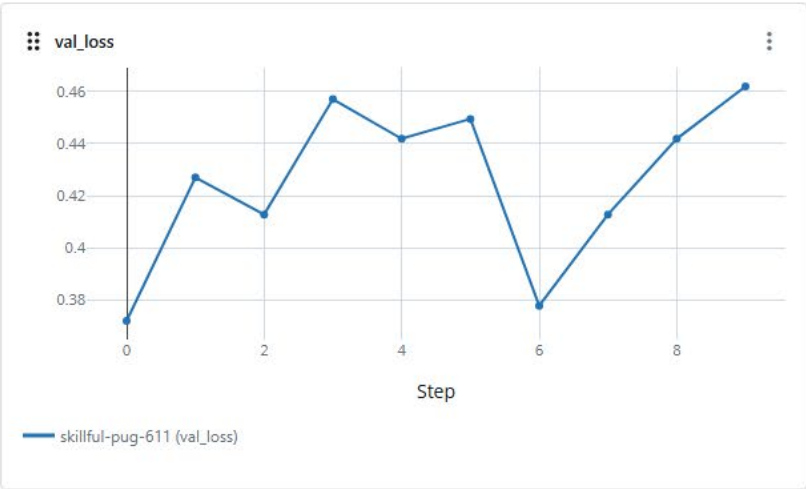
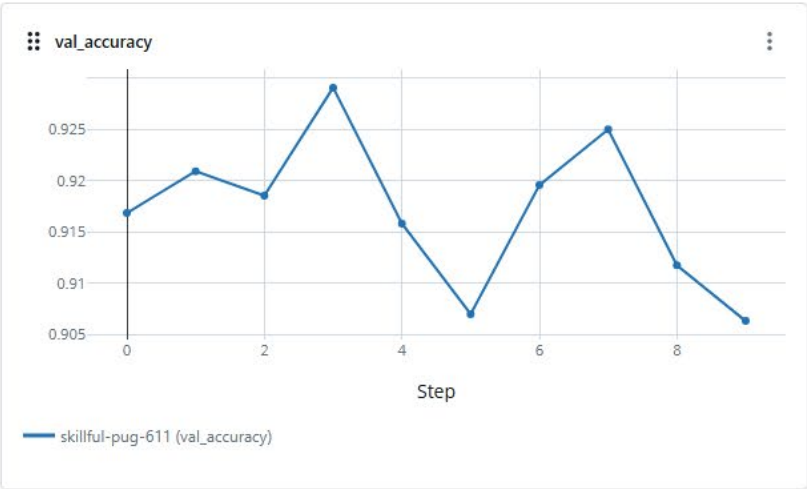
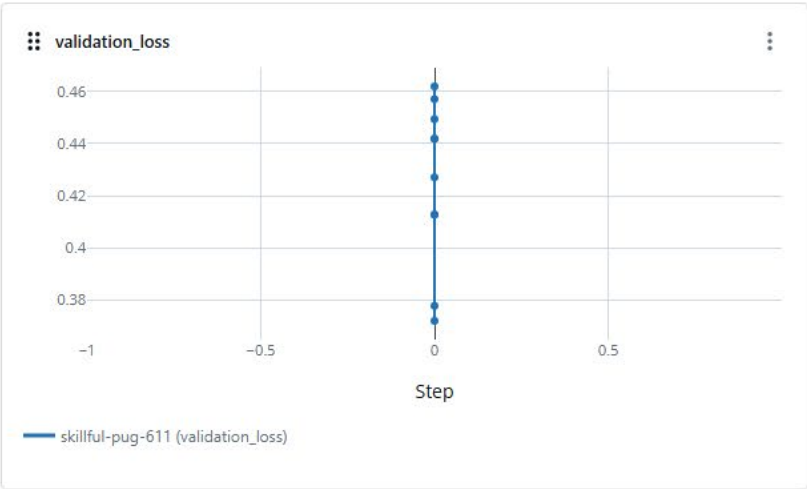
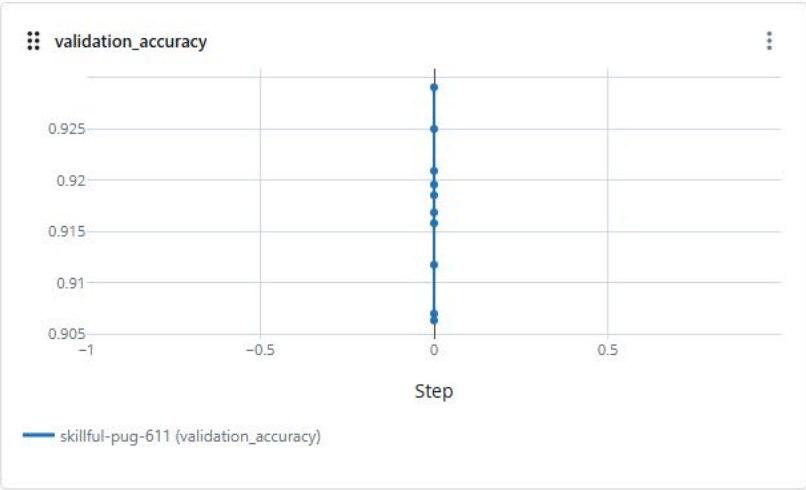
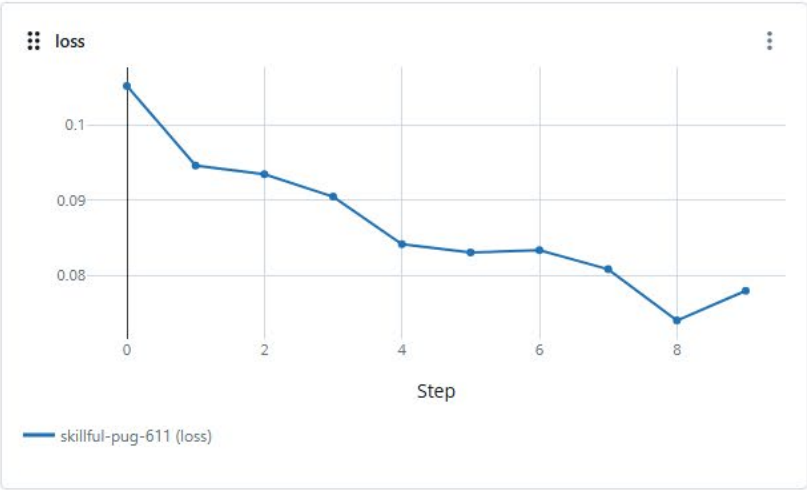
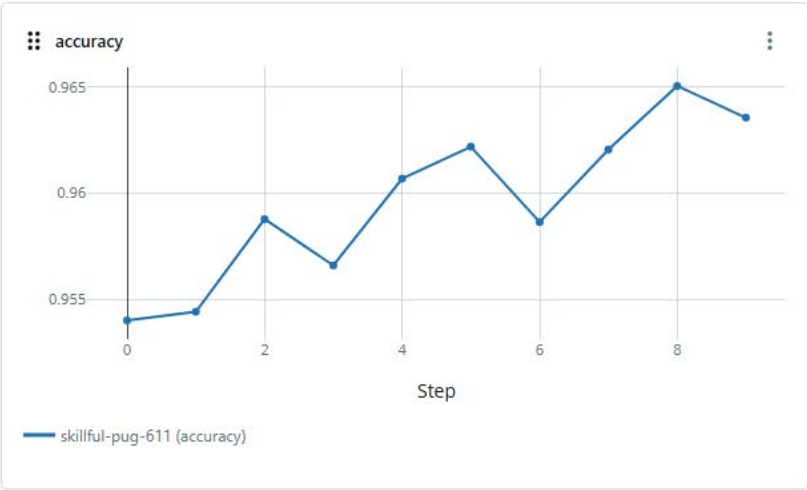
MLFlow Tracking User Interface

Ejemplo con MLflow >

skillful-pug-611

Overview **Model metrics** System metrics Traces Artifacts

Refresh



MLFlow Tracking User Interface

Parameters (15)

Search parameters

Parameter	Value
optimizer_amsg rad	False
optimizer_beta_1	0.9
optimizer_beta_2	0.999
optimizer_clipnorm	None
optimizer_clipvalue	None
optimizer_ema_momentum	0.99
optimizer_ema_overwrite_frequency	None
optimizer_epsilon	1e-07
optimizer_global_clipnorm	None
optimizer_gradient_accumulation_steps	None
optimizer_learning_rate	0.00100000000474974513
optimizer_loss_scale_factor	None
optimizer_name	adam
optimizer_use_ema	False
optimizer_weight_decay	None

Metrics (6)

Search metrics

Metric	Value
accuracy	0.9635473489761353
loss	0.07794398069381714
validation_accuracy	0.9063454270362854
validation_loss	0.46198078989982605
val_accuracy	0.9063454270362854
val_loss	0.46198078989982605

model_summary.txt 3.78KB

Path: mlflow-artifacts:/411331234257361705/5e71bf2e06b84105a6e1c06675950678/artifact

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 126, 64)	1,792
max_pooling1d (MaxPooling1D)	(None, 63, 64)	0
dropout (Dropout)	(None, 63, 64)	0
conv1d_1 (Conv1D)	(None, 61, 128)	24,704
max_pooling1d_1 (MaxPooling1D)	(None, 30, 128)	0
dropout_1 (Dropout)	(None, 30, 128)	0
flatten (Flatten)	(None, 3840)	0
dense (Dense)	(None, 100)	384,100
dropout_2 (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 6)	606

Total params: 1,233,608 (4.71 MB)

Trainable params: 411,202 (1.57 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 822,406 (3.14 MB)